computational-biology: the Computational Biology program — user guide

computational-biology-ai999x 1.0.3, Rust build: command line, desktop app and browser. **Creator / engineer:** Ibnul Jaif Farabi · **Company:** AI999X · support@ai999x.com

Computational Biology covers DNA and protein sequences, alignment, molecular weights, genetics and population genetics, phylogenetics, enzyme kinetics, metabolic and population models, and ecology. This guide is about `computational-biology`, the model as a self-contained program for Windows and Linux: 83 capabilities in 10 topics, answered on your own computer with no network, no Python and nothing else to install. The same compiled model runs in the AI999X desktop app and in the browser on the model's page (https://www.ai999x.com/models/computational-biology-ai999x/); the Python package (`pip install computational-biology-ai999x`) has its own guide. Every command output printed here is what the program printed.

Contents

- 1. Get it: desktop app, command line, browser, Python
- 2. Quick start
- 3. Writing inputs
- 4. Outcomes and exit codes
- 5. Scripts and agents: batch and JSON
- 6. Figures
- 7. In a web page or Node.js: WebAssembly
- 8. Every capability

1. Get it

Version 1.4.0 of the AI999X desktop app and of the command-line bundle. Checksums for every file: https://www.ai999x.com/downloads/desktop/1.4.0/SHA256SUMS; the newest version is always under https://www.ai999x.com/downloads/desktop/latest/. Full instructions: https://www.ai999x.com/desktop-cli/

what	system	download
Desktop app: Computational Biology	Windows x86-64	https://www.ai999x.com/downloads/desktop/1.4.0/ai999x-computational-biology-1.4.0-windows-x86_64.zip
Desktop app: Computational Biology	Linux x86-64	https://www.ai999x.com/downloads/desktop/1.4.0/ai999x-computational-biology-1.4.0-linux-x86_64.tar.gz
Desktop app: all models	Windows x86-64	https://www.ai999x.com/downloads/desktop/1.4.0/ai999x-1.4.0-windows-x86_64.zip
Desktop app: all models	Linux x86-64	https://www.ai999x.com/downloads/desktop/1.4.0/ai999x-1.4.0-linux-x86_64.tar.gz
Command line	Windows x86-64	https://www.ai999x.com/downloads/desktop/1.4.0/ai999x-cli-1.4.0-windows-x86_64.zip
Command line	Linux x86-64	https://www.ai999x.com/downloads/desktop/1.4.0/ai999x-cli-1.4.0-linux-x86_64.tar.gz

Desktop app

Computational Biology in its own app: search, questions in words, input forms, live figures and explanations. Windows: Download ai999x-computational-biology-1.4.0-windows-x86_64.zip, right-click it → Extract All, then open the folder and start ai999x-computational-biology.exe. If Windows SmartScreen asks, choose More info → Run anyway. Linux:

```
curl -LO https://www.ai999x.com/downloads/desktop/1.4.0/ai999x-computational-biology-1.4.0-linux-x86_64.tar.gz
curl -LO https://www.ai999x.com/downloads/desktop/1.4.0/SHA256SUMS
sha256sum -c --ignore-missing SHA256SUMS
tar xzf ai999x-computational-biology-1.4.0-linux-x86_64.tar.gz
./ai999x-computational-biology-1.4.0-linux-x86_64/ai999x-computational-biology
```

It opens on this model: type a question or pick a capability, fill in the form and press Compute. The AI999X desktop app with every model works the same way (choose the model first). Windows: Download ai999x-1.4.0-windows-x86_64.zip, right-click it → Extract All, then open the folder and start ai999x.exe. If Windows SmartScreen asks, choose More info → Run anyway. Linux:

```
curl -LO https://www.ai999x.com/downloads/desktop/1.4.0/ai999x-1.4.0-linux-x86_64.tar.gz
curl -LO https://www.ai999x.com/downloads/desktop/1.4.0/SHA256SUMS
sha256sum -c --ignore-missing SHA256SUMS
tar xzf ai999x-1.4.0-linux-x86_64.tar.gz
./ai999x-1.4.0-linux-x86_64/ai999x
```

Command line — Linux

The bundle holds one program per model; `computational-biology` is this one. The checksum line should print OK.

```
curl -LO https://www.ai999x.com/downloads/desktop/1.4.0/ai999x-cli-1.4.0-linux-x86_64.tar.gz
curl -LO https://www.ai999x.com/downloads/desktop/1.4.0/SHA256SUMS
sha256sum -c --ignore-missing SHA256SUMS
mkdir -p ~/.local/share && tar xzf ai999x-cli-1.4.0-linux-x86_64.tar.gz -C ~/.local/share
echo 'export PATH="$HOME/.local/share/ai999x-cli-1.4.0-linux-x86_64:$PATH"' >> ~/.bashrc && source ~/.bashrc
computational-biology --version
```

Command line — Windows (PowerShell)

The fourth line should print True.

```
$F = "ai999x-cli-1.4.0-windows-x86_64.zip"
Invoke-WebRequest "https://www.ai999x.com/downloads/desktop/1.4.0/$F" -OutFile $F
Invoke-WebRequest "https://www.ai999x.com/downloads/desktop/1.4.0/SHA256SUMS" -OutFile SHA256SUMS
(Get-FileHash $F -Algorithm SHA256).Hash.ToLower() -eq ((Select-String -SimpleMatch $F SHA256SUMS).Line -split ' ')[0]
Expand-Archive $F -DestinationPath "$env:LOCALAPPDATA\AI999X" -Force
$D = "$env:LOCALAPPDATA\AI999X\ai999x-cli-1.4.0-windows-x86_64"; $env:Path += ";$D"
[Environment]::SetEnvironmentVariable("Path", [Environment]::GetEnvironmentVariable("Path", "User") + ";$D", "User")
computational-biology --version
```

In your browser

Open https://www.ai999x.com/models/computational-biology-ai999x/, choose a capability and press Compute in your browser: the same program runs on your device as WebAssembly; nothing is sent anywhere.

Python

```
pip install computational-biology-ai999x
```

2. Quick start

2.1 Check the install

Prints the model, its version and that this is the Rust build.

```
$ computational-biology --version
computational-biology-ai999x 1.0.3 (Rust build, ax-engine 0.1.0)
```

2.2 Ask in words

Write the question as you would say it: the program reads the quantities and their units, picks the capability and answers.

```
$ computational-biology ask "Reverse complement: seq = ATGCCGTA"
Read as: Reverse complement of a DNA sequence
revcomp = TACGGCAT
  by: Reverse complement of a DNA sequence  (complement each base (A-T, C-G) and reverse the order)
  seq = ATGCCGTA
  assumes: DNA letters A, C, G, T only
```

2.3 Find a capability

Search the capabilities by what you want to know; the first column is the id to pass to info and solve.

```
$ computational-biology search "hardy-weinberg chi-square test"
bio_hardy_weinberg_test              Hardy-Weinberg chi-square test  (population genetics / Hardy-Weinberg)
bio_hardy_weinberg_exact_test        Hardy-Weinberg exact test  (population genetics / Hardy-Weinberg)
bio_michaelis_menten_fit             Michaelis-Menten nonlinear least-squares fit  (enzyme kinetics / Michaelis-Menten)
bio_hill_fit                         Hill equation nonlinear least-squares fit  (enzyme kinetics / cooperativity)
bio_tajima_d                         Tajima's D  (population genetics / neutrality tests)
bio_linkage_disequilibrium           Linkage disequilibrium D, D' and r^2  (population genetics / linkage disequilibrium)
bio_pedigree_inbreeding              Inbreeding coefficient from a pedigree  (genetics / inbreeding)
bio_semiglobal_alignment_score       Semi-global (end-gap-free) alignment score  (alignment / pairwise)
… 6 more
```

2.4 List the capabilities

Every capability the program answers: its id (what you pass to solve) and its name.

```
$ computational-biology list
bio_ace                                ACE species richness estimator
bio_amino_acid_composition             Amino-acid composition of a protein
bio_aromaticity                        Aromaticity of a protein
bio_beta_diversity                     Beta diversity between two samples
bio_brillouin_diversity                Brillouin diversity index
bio_chao1                              Chao1 species richness estimator
bio_chemostat_steady_state             Chemostat steady state (Monod kinetics)
bio_coalescent_times                   Expected coalescence times
… 75 more
```

2.5 What a capability needs

The equation or method, every input and output with its SI unit. Read this before solve: it tells you the input names.

```
$ computational-biology info bio_hardy_weinberg_test
Hardy-Weinberg chi-square test
  allele counting, expected counts N p^2, 2Npq, N q^2, chi-square with 3 - 1 - 1 = 1 degree of freedom
  in   n_AA     number of AA individuals (integer) [1]
  in   n_Aa     number of Aa individuals (integer) [1]
  in   n_aa     number of aa individuals (integer) [1]
  out  p        frequency of allele A [1]
  out  q        frequency of allele a [1]
  out  exp_AA   expected AA count (N p^2) [1]
  out  exp_Aa   expected Aa count (2 N p q) [1]
  out  exp_aa   expected aa count (N q^2) [1]
  out  chi2     chi-square statistic [1]
  out  p_value  p-value (chi-square, 1 degree of freedom) [1]
  out  Ho       observed heterozygosity [1]
  out  He       expected heterozygosity 2pq [1]
  out  F        inbreeding coefficient 1 - Ho/He [1]
```

2.6 Solve

Inputs as name=value, each with any unit of the right kind. The answer lists every output, the method used, each input converted to SI, and the conditions the answer assumes.

```
$ computational-biology solve bio_hardy_weinberg_test n_AA=298 n_Aa=489 n_aa=213
p = 0.5425
q = 0.4575
exp_AA = 294.30625
exp_Aa = 496.3875
exp_aa = 209.30625
chi2 = 0.2214895876
p_value = 0.6379072638
Ho = 0.489
He = 0.4963875
F = 0.01488252625
  by: Hardy-Weinberg chi-square test  (allele counting, expected counts N p^2, 2Npq, N q^2, chi-square with 3 - 1 - 1 = 1 degree
of freedom)
  assumes: both alleles present; biallelic locus; no continuity correction
```

2.7 Machine-readable answer

--json prints the same answer as one line of JSON (values in SI): for scripts and other programs.

```
$ computational-biology solve bio_hardy_weinberg_test n_AA=298 n_Aa=489 n_aa=213 --json
{"outcome":"SOLVED","message":"","value":0.221489587647,"unit":"","solved_for":"chi2","values":[["p",0.5425,""],["q",0.4575,""],
["exp_AA",294.30625,""],["exp_Aa",496.3875,""],["exp_aa",209.30625,""],["chi2",0.221489587647,""],["p_value",0.637907263782,""],
["Ho",0.489,""],["He",0.4963875,""],["F",0.0148825262522,""]],"target":"chi2","phrase":"frequency of allele A","assumptions":
["both alleles present","biallelic locus","no continuity correction"],"name":"Hardy-Weinberg chi-square test","equation":"allele
counting, expected counts N p^2, 2Npq, N q^2, chi-square with 3 - 1 - 1 = 1 degree of freedom"}
```

2.8 When an input is missing

No number is guessed: the answer names what is missing, shows a complete call, and the program exits with status 1.

```
$ computational-biology solve bio_hardy_weinberg_test n_AA=298 n_Aa=489
NEEDS_INPUT: bio_hardy_weinberg_test needs n_aa (number of aa individuals). Example: solve('bio_hardy_weinberg_test', n_AA=298,
n_Aa=489, n_aa=213).
```

2.9 Draw the figures

The live figures for your own inputs, written as SVG files into the folder given with --out (here the current folder); open them in any browser. Without --out the SVG text comes back as JSON.

```
$ computational-biology figures bio_reverse_complement seq=ATGCCGTA --out .
[
  {
    "slug": "bio-reverse-complement-duplex",
    "title": "Reverse complement: the partner strand of a DNA sequence",
    "caption": "The given strand ATGCCGTA (top, 5' to 3') paired with its partner strand (bottom, 3' to 5'); A-T and G-C pairs in
two colours. Read 5' to 3', the partner strand is the model's reverse complement, revcomp = TACGGCAT. Your inputs: seq =
ATGCCGTA.",
    "file": ".\\bio-reverse-complement-duplex.svg"
  },
  {
    "slug": "bio-reverse-complement-live-1",
    "title": "How far each letter of seq moves",
    "caption": "For each letter of seq and the same position of revcomp: how many places along the alphabet (A to Z, wrapping
round) it moved. Your inputs: seq = ATGCCGTA.",
    "file": ".\\bio-reverse-complement-live-1.svg"
  }
]
```

2.10 Many questions at once

One JSON request per line in, one JSON answer per line out, in order: for scripts, notebooks and agents (the program starts once). In PowerShell: Get-Content cases.jsonl | computational-biology batch

`cases.jsonl`:

```
{"key": "bio_hardy_weinberg_test", "inputs": {"n_AA": "298", "n_Aa": "489", "n_aa": "213"}}
{"key": "bio_reverse_complement", "inputs": {"seq": "ATGCCGTA"}}
```

```
$ computational-biology batch < cases.jsonl
{"outcome":"SOLVED","message":"","value":0.221489587647,"unit":"","solved_for":"chi2","values":[["p",0.5425,""],["q",0.4575,""],
["exp_AA",294.30625,""],["exp_Aa",496.3875,""],["exp_aa",209.30625,""],["chi2",0.221489587647,""],["p_value",0.637907263782,""],
["Ho",0.489,""],["He",0.4963875,""],["F",0.0148825262522,""]],"target":"chi2","phrase":"frequency of allele A","assumptions":
["both alleles present","biallelic locus","no continuity correction"],"name":"Hardy-Weinberg chi-square test","equation":"allele
counting, expected counts N p^2, 2Npq, N q^2, chi-square with 3 - 1 - 1 = 1 degree of freedom"}
{"outcome":"SOLVED","message":"","value":null,"unit":"","solved_for":"revcomp","values":
[["revcomp","TACGGCAT",""]],"target":"revcomp","phrase":"reverse complement 5'->3'","assumptions":["DNA letters A, C, G, T
only"],"name":"Reverse complement of a DNA sequence","equation":"complement each base (A-T, C-G) and reverse the order"}
```

3. Writing inputs

Inputs are written `name=value`, one per argument; `computational-biology info <capability>` lists the names. Quote an argument that holds a space or brackets (bash, zsh and PowerShell all accept double quotes). The answer always states each input as it was read, converted to SI, so a misread unit is visible at once.

kind	example	how it is read
a number with a unit	`"R=0.128 nm"`	any unit of the same kind; converted to SI
a number in SI	`R=1.28e-10`	no unit means the SI unit shown by info
a choice	`structure=FCC`	one of the values info lists
a list or vector	`"v=[1, 2, 3]" or "v=1, 2, 3"`	brackets keep it a list
a matrix	`"A=[[1, 2], [3, 4]]"`	one list per row
true / false	`inclusive=true`	for yes/no options

An equation can be solved for any of its quantities: give the others and add `--target <symbol>`. Without `--target`, the program solves for the quantity the capability is named after.

4. Outcomes and exit codes

The program never guesses a number: when it cannot answer, it says why and exits with status 1, so a script can tell an answer from a refusal without reading the text.

outcome	meaning	exit status
SOLVED	the answer: every output with its unit, the method, the inputs in SI, the conditions assumed	0
NEEDS_INPUT	an input is missing; the message names it and shows a complete call	1
INVALID_INPUT	a value or unit of the wrong kind, an unknown input name, or a value outside the conditions	1
AMBIGUOUS	the inputs allow two different answers; say which one is meant	1
OUT_OF_SCOPE	the model has no capability for this, or the result would not be meaningful	1
(usage)	a command the program does not know: it prints its usage	2

5. Scripts and agents: batch and JSON

`computational-biology solve ... --json` prints one answer as one line of JSON. `computational-biology batch` reads one request per line on standard input and writes one JSON answer per line, in order, starting the program once for any number of requests. A request is `{"key": "`

`<capability>", "inputs": {"name": "value", ...}, "target": null}`. In PowerShell, pipe the file in: `Get-Content cases.jsonl | computational-biology batch`. `computational-biology catalog` prints every capability with its inputs as JSON, to build forms or tool definitions.

answer field	meaning
outcome	one of the outcomes above
message	why there is no answer (empty when SOLVED)
value, unit	the main result in SI units
values	every output: [symbol, value, unit] (procedures)
target	the quantity solved for
si	each numeric input converted to SI: [symbol, value]
assumptions	the conditions the answer relies on
name, equation	the capability and its equation or method

```
import json, subprocess

reqs = [{"key": "<capability>", "inputs": {"name": "value"}}]
out = subprocess.run(["computational-biology", "batch"], input="\n".join(map(json.dumps, reqs)),
                     capture_output=True, text=True, check=True).stdout
answers = [json.loads(line) for line in out.splitlines()]
```

6. Figures

`computational-biology figures <capability> name=value ... --out DIR` draws the live figures for those inputs and writes each as an SVG file into DIR (an existing folder); without `--out` the SVG text comes back as JSON. The desktop app and the model page draw the same figures as soon as you press Compute.

```
$ computational-biology figures bio_reverse_complement seq=ATGCCGTA --out .
[
  {
    "slug": "bio-reverse-complement-duplex",
    "title": "Reverse complement: the partner strand of a DNA sequence",
    "caption": "The given strand ATGCCGTA (top, 5' to 3') paired with its partner strand (bottom, 3' to 5'); A-T and G-C pairs in
two colours. Read 5' to 3', the partner strand is the model's reverse complement, revcomp = TACGGCAT. Your inputs: seq =
ATGCCGTA.",
    "file": ".\\bio-reverse-complement-duplex.svg"
  },
  {
    "slug": "bio-reverse-complement-live-1",
    "title": "How far each letter of seq moves",
    "caption": "For each letter of seq and the same position of revcomp: how many places along the alphabet (A to Z, wrapping
round) it moved. Your inputs: seq = ATGCCGTA.",
    "file": ".\\bio-reverse-complement-live-1.svg"
  }
]
```

7. In a web page or Node.js: WebAssembly

The same model is a WebAssembly module, `computational_biology_ai999x.wasm` (https://www.ai999x.com/compute/rust/computational_biology_ai999x.wasm), with a small loader, `ax-wasm.js` (https://www.ai999x.com/compute/rust/ax-wasm.js). It runs in a browser, a Web Worker or Node.js, with no server.

```
<script src="ax-wasm.js"></script>
<script>
  AXWasm.load("computational_biology_ai999x.wasm").then(function (m) {
    var a = m.solve({key: "<capability>", inputs: {name: "value"}});
    console.log(a.outcome, a.values || a.value);
    console.log(m.catalog().capabilities.length + " capabilities");
  });
</script>
```

8. Every capability

By topic. For each: what it does, what it needs and gives, when it applies, and a command with what the program printed for it. `computational-biology info <id>` shows the same inputs and outputs in the terminal.

Alignment (7)

Global alignment score (Needleman-Wunsch, linear gaps) — `bio_global_alignment_score`

The optimal global (Needleman-Wunsch) alignment score of two sequences with stated match, mismatch and linear gap scores.

$$S_{i,j} = \max(S_{i-1,j-1} + s(x_i, y_j),\ S_{i-1,j} + g,\ S_{i,j-1} + g)$$

symbol		meaning	SI unit
`seq1`	in	first sequence	—
`seq2`	in	second sequence	—
`match`	in	score of a match	—
`mismatch`	in	score of a mismatch	—
`gap`	in	score of each gap position (<= 0)	—
`score`	out	optimal global alignment score	—

Applies when: gap <= 0; end gaps scored like internal gaps.

```
$ computational-biology solve bio_global_alignment_score seq1=GATTACA seq2=GCATGCU match=1 mismatch=-1 gap=-1
score = 0
  by: Global alignment score (Needleman-Wunsch, linear gaps)  (dynamic programming over prefixes (Needleman-Wunsch))
  match = 1
  mismatch = -1
  gap = -1
  seq1 = GATTACA
  seq2 = GCATGCU
  assumes: gap <= 0; end gaps scored like internal gaps
```

What the worked example means: This calculation determines the highest possible score for aligning two entire sequences end-to-end. By applying dynamic programming to the prefixes of sequence one and sequence two, it evaluates every possible alignment path. The result, 0, represents the sum of scores for matched characters, mismatched characters, and gap positions, where each gap contributes a fixed penalty defined by the input parameter. This metric quantifies the overall similarity between the two inputs under the specified scoring scheme.

Global alignment score with affine gaps (Gotoh) — `bio_global_alignment_affine`

The optimal global alignment score with affine gaps: a gap of length L scores open + (L-1)*extend.

$$g(L) = o + (L-1)\,e$$

symbol		meaning	SI unit
`seq1`	in	first sequence	—
`seq2`	in	second sequence	—
`match`	in	score of a match	—
`mismatch`	in	score of a mismatch	—
`open`	in	score of the first position of a gap (<= 0)	—
`extend`	in	score of each further gap position (<= 0)	—
`score`	out	optimal global alignment score	—

Applies when: open, extend <= 0; a gap of length L scores open + (L-1) extend; end gaps scored.

```
$ computational-biology solve bio_global_alignment_affine seq1=ACGTTTACG seq2=ACGACG match=2 mismatch=-1 open=-5 extend=-1
score = 5
  by: Global alignment score with affine gaps (Gotoh)  (Gotoh's three-state dynamic programming)
  match = 2
  mismatch = -1
  open = -5
  extend = -1
  seq1 = ACGTTTACG
  seq2 = ACGACG
  assumes: open, extend <= 0; a gap of length L scores open + (L-1) extend; end gaps scored
```

What the worked example means: This calculation determines the highest possible similarity score for aligning two biological sequences globally, treating a new gap opening differently from extending an existing one. By applying Gotoh's dynamic programming method with a match score of 2 and mismatch penalty of -1, the algorithm evaluates all possible alignments while penalizing gaps based on an open cost of -5 and an extension cost of -1. The resulting value of 5 represents the maximum cumulative score achievable under these specific scoring rules, accounting for both substitutions and indels across the entire length of the sequences.

Global protein alignment score (BLOSUM62, affine gaps) — `bio_protein_global_blosum62`

The optimal global alignment score of two proteins with BLOSUM62 and affine gaps.

$$S = \max_{\text{alignments}} \sum B_{62}(x,y) + \sum_{\text{gaps}} [o + (L-1)e]$$

symbol		meaning	SI unit
`seq1`	in	first protein sequence	—
`seq2`	in	second protein sequence	—
`open`	in	score of the first position of a gap (<= 0)	—
`extend`	in	score of each further gap position (<= 0)	—

symbol		meaning	SI unit
`score`	out	optimal global alignment score	—

Applies when: standard amino acids; open, extend <= 0.

```
$ computational-biology solve bio_protein_global_blosum62 seq1=HEAGAWGHEE seq2=PAWHEAE open=-10 extend=-1
score = 3
  by: Global protein alignment score (BLOSUM62, affine gaps)  (Gotoh's dynamic programming with BLOSUM62 substitution scores)
  open = -10
  extend = -1
  seq2 = PAWHEAE
  seq1 = HEAGAWGHEE
  assumes: standard amino acids; open, extend <= 0
```

What the worked example means: This calculation determines the best global alignment score between two protein sequences using the BLOSUM62 substitution matrix and affine gap penalties. The result, 3, represents the maximum sum of match scores plus gap costs when aligning HEAGAWGHEE with PAWHEAE from end to end. A higher value indicates greater similarity and a better fit under the chosen scoring scheme.

Local alignment score (Smith-Waterman, linear gaps) — `bio_local_alignment_score`

The optimal local (Smith-Waterman) alignment score of two sequences with stated match, mismatch and linear gap scores.

$$S_{i,j} = \max(0,\ S_{i-1,j-1} + s(x_i, y_j),\ S_{i-1,j} + g,\ S_{i,j-1} + g)$$

symbol		meaning	SI unit
`seq1`	in	first sequence	—
`seq2`	in	second sequence	—
`match`	in	score of a match	—
`mismatch`	in	score of a mismatch	—
`gap`	in	score of each gap position (<= 0)	—
`score`	out	optimal local alignment score (>= 0)	—

Applies when: match > 0; gap <= 0.

```
$ computational-biology solve bio_local_alignment_score seq1=TGTTACGG seq2=GGTTGACTA match=3 mismatch=-3 gap=-2
score = 13
  by: Local alignment score (Smith-Waterman, linear gaps)  (dynamic programming with a floor at 0 (Smith-Waterman))
  match = 3
  mismatch = -3
  gap = -2
  seq1 = TGTTACGG
  seq2 = GGTTGACTA
  assumes: match > 0; gap <= 0
```

What the worked example means: This calculation determines the highest possible score for aligning a specific segment from the first sequence with a segment from the second sequence. Using dynamic programming, it evaluates matches, mismatches, and gaps to find the best local similarity. The resulting score of 13 indicates the strength of this local relationship, where higher values suggest more significant biological homology or shared motifs.

Local alignment score with affine gaps — `bio_local_alignment_affine`

The optimal local alignment score with affine gaps (a gap of length L scores open + (L-1)*extend).

$$g(L) = o + (L-1)\,e, \quad S \geq 0$$

symbol		meaning	SI unit
`seq1`	in	first sequence	—
`seq2`	in	second sequence	—
`match`	in	score of a match	—
`mismatch`	in	score of a mismatch	—
`open`	in	score of the first position of a gap (<= 0)	—
`extend`	in	score of each further gap position (<= 0)	—
`score`	out	optimal local alignment score (>= 0)	—

Applies when: match > 0; open, extend <= 0.

```
$ computational-biology solve bio_local_alignment_affine seq1=TTACGTACGTTT seq2=GGACGTTACGTGG match=2 mismatch=-1 open=-3
extend=-1
score = 13
  by: Local alignment score with affine gaps  (Gotoh's recursion with a floor at 0)
  match = 2
  mismatch = -1
  open = -3
  extend = -1
  seq2 = GGACGTTACGTGG
  seq1 = TTACGTACGTTT
  assumes: match > 0; open, extend <= 0
```

What the worked example means: This calculation determines the best local alignment score using Gotoh's algorithm, which models gaps with affine penalties. The result, 13, represents the highest possible similarity between the two input sequences under the specified scoring scheme. This method assumes that opening a gap costs more than extending it, reflecting biological reality where insertions or deletions often occur as contiguous blocks rather than scattered single events. The score is non-negative due to the local alignment constraint, meaning any negative cumulative score resets to zero.

Local protein alignment score (BLOSUM62, affine gaps) — `bio_protein_local_blosum62`

The optimal local (Smith-Waterman) alignment score of two proteins with BLOSUM62 and affine gaps.

$$S = \max\left(0, \max_{\text{local alignments}} \sum B_{62} + \sum[o + (L-1)e]\right)$$

symbol		meaning	SI unit
`seq1`	in	first protein sequence	—
`seq2`	in	second protein sequence	—
`open`	in	score of the first position of a gap (<= 0)	—
`extend`	in	score of each further gap position (<= 0)	—
`score`	out	optimal local alignment score (>= 0)	—

Applies when: standard amino acids; open, extend <= 0.

```
$ computational-biology solve bio_protein_local_blosum62 seq1=HEAGAWGHEE seq2=PAWHEAE open=-8 extend=-8
score = 20
  by: Local protein alignment score (BLOSUM62, affine gaps)  (Gotoh's recursion with BLOSUM62 scores and a floor at 0)
  open = -8
  extend = -8
  seq2 = PAWHEAE
  seq1 = HEAGAWGHEE
  assumes: standard amino acids; open, extend <= 0
```

What the worked example means: This calculation determines the highest-scoring local similarity between two protein sequences using BLOSUM62 substitution scores and affine gap penalties. The result, 20, represents the maximum sum of match scores minus gap costs for any contiguous subsequence pair. A positive value indicates significant homology or conserved motifs, while a zero suggests no detectable local similarity above background noise.

Semi-global (end-gap-free) alignment score — `bio_semiglobal_alignment_score`

The optimal alignment score when gaps at either end of either sequence are free (fitting a read inside a longer sequence, overlaps), with linear internal gaps.

$$S = \max\left(\max_j S_{n,j},\ \max_i S_{i,m}\right),\ S_{i,0} = S_{0,j} = 0$$

symbol		meaning	SI unit
`seq1`	in	first sequence	—
`seq2`	in	second sequence	—
`match`	in	score of a match	—
`mismatch`	in	score of a mismatch	—
`gap`	in	score of each internal gap position (<= 0)	—
`score`	out	optimal semi-global alignment score	—

Applies when: gap <= 0; end gaps unscored.

```
$ computational-biology solve bio_semiglobal_alignment_score seq1=TTTTACGTACGGGG seq2=ACGTAC match=1 mismatch=-1 gap=-1
score = 6
  by: Semi-global (end-gap-free) alignment score  (Needleman-Wunsch with a zero first row and column and the best score on the
last row or column)
  match = 1
  mismatch = -1
  gap = -1
  seq1 = TTTTACGTACGGGG
  seq2 = ACGTAC
  assumes: gap <= 0; end gaps unscored
```

What the worked example means: This calculation determines the highest possible alignment score between two sequences where gaps at the beginning and end are not penalized. Using a scoring scheme that assigns 1 for matches, -1 for mismatches, and -1 for internal gaps, the algorithm identifies the best overlap or containment scenario. The resulting score of 6 represents the quality of this alignment under conditions where terminal gaps are free.

Ecology (14)

ACE species richness estimator — `bio_ace`

The abundance-based coverage estimator (ACE) of species richness, with a stated rare-species threshold (10 is the usual value).

$$S_{ace} = S_{abund} + \frac{S_{rare}}{C_{ace}} + \frac{F_1}{C_{ace}}\gamma^2$$

symbol		meaning	SI unit
`counts`	in	number of individuals of each species	—
`rare_threshold`	in	largest count of a rare species (usually 10)	—
`ace`	out	estimated species richness	—

Applies when: whole counts; not every rare species a singleton.

```
$ computational-biology solve bio_ace "counts=[1, 1, 2, 3, 5, 8, 20, 50]" rare_threshold=10
ace = 9.719298246
  by: ACE species richness estimator  (abundant species counted directly; rare species scaled by the sample coverage C = 1 -
F1/N_rare with the coefficient-of-variation term (EstimateS form, as scikit-bio))
  assumes: whole counts; not every rare species a singleton
```

What the worked example means: This calculation determines the abundance-based coverage estimator of species richness. It separates observed species into abundant and rare groups based on a threshold count, typically ten individuals. The method uses the number of abundant species plus an adjustment for rare species that accounts for sample coverage and the variance in their frequencies. The final estimate is 9.7193, which represents the predicted total number of distinct species in the community given the sampling effort.

Berger-Parker and McIntosh indices — `bio_dominance_indices`

The Berger-Parker dominance (share of the most abundant species) and McIntosh's index of a sample.

$$d = \frac{n_{\max}}{N}, \quad D = \frac{N - U}{N - \sqrt{N}}$$

symbol		meaning	SI unit
`counts`	in	number of individuals of each species	—
`berger_parker`	out	Berger-Parker dominance n_max/N	—
`mcintosh`	out	McIntosh index (N - U)/(N - sqrt N), U = sqrt(sum n_i^2)	—

Applies when: whole counts; N >= 2.

```
$ computational-biology solve bio_dominance_indices "counts=[50, 30, 20]"
berger_parker = 0.5
mcintosh = 0.4261762219
  by: Berger-Parker and McIntosh indices  (largest count over N; McIntosh's U statistic)
  assumes: whole counts; N >= 2
```

What the worked example means: This calculation determines community dominance using two metrics derived from species counts. The Berger-Parker index represents the proportion of the total individuals that belong to the most abundant species, resulting in a value of 0.5. The McIntosh index measures diversity based on the distance of the community from an even distribution, yielding a score of 0.426176. These values indicate how much a single taxon controls the sample structure.

Beta diversity between two samples — `bio_beta_diversity`

The Bray-Curtis dissimilarity (abundances) and the Jaccard and Sorensen dissimilarities (presence and absence) between two community samples.

$$BC = \frac{\sum |x_i - y_i|}{\sum (x_i + y_i)}, \quad J = 1 - \frac{|A \cap B|}{|A \cup B|}$$

symbol		meaning	SI unit
`x`	in	abundances of each species in the first sample	—
`y`	in	abundances of the same species in the second sample	—
`bray_curtis`	out	Bray-Curtis dissimilarity	—
`jaccard`	out	Jaccard dissimilarity (1 - shared/union)	—
`sorensen`	out	Sorensen dissimilarity (1 - 2 shared/(S_x + S_y))	—

Applies when: the same species order in both samples; each sample non-empty.

```
$ computational-biology solve bio_beta_diversity "x=[6, 7, 4, 0]" "y=[10, 0, 6, 3]"
bray_curtis = 0.4444444444
jaccard = 0.5
sorensen = 0.3333333333
  by: Beta diversity between two samples  (sum |x - y| / sum (x + y); presence-absence set differences)
  assumes: the same species order in both samples; each sample non-empty
```

What the worked example means: This calculation determines the compositional difference between two ecological communities using abundance and presence-absence data. The Bray-Curtis dissimilarity, which measures how much the relative abundances of shared species differ, resulted in 0.444444. Additionally, the Jaccard dissimilarity, reflecting the proportion of species unique to either sample relative to the total union, was calculated as 0.5. These values quantify the extent to which the two samples are distinct from one another based on their species profiles.

Brillouin diversity index — `bio_brillouin_diversity`

Brillouin's diversity index of a fully censused collection (natural logarithm).

$$H_B = \frac{\ln N! - \sum_i \ln n_i!}{N}$$

symbol		meaning	SI unit
`counts`	in	number of individuals of each species	—
`H_B`	out	Brillouin index (nats per individual)	—

Applies when: whole counts; N <= 20000.

```
$ computational-biology solve bio_brillouin_diversity "counts=[5, 3, 2]"
H_B = 0.7832014181
  by: Brillouin diversity index  (logarithm of the multinomial coefficient over N (exact integer))
  assumes: whole counts; N <= 20000
```

What the worked example means: This calculation determines the Brillouin diversity index, a measure of species diversity in a fully enumerated population. It uses the natural logarithm of the multinomial coefficient divided by the total number of individuals. The result, 0.783201, represents the information content per individual in nats. This value is derived from the specific counts of each species provided as [5, 3, 2]. Because this method assumes a complete census rather than a random sample, it describes the actual diversity of the observed group without estimating parameters for a larger unseen population.

Chao1 species richness estimator — `bio_chao1`

The Chao1 estimate of total species richness from abundance counts (bias-corrected or classic form), with the observed richness and the numbers of singletons and doubletons.

$$\hat{S} = S_{obs} + \frac{F_1(F_1 - 1)}{2(F_2 + 1)}$$

symbol		meaning	SI unit
`counts`	in	number of individuals of each species	—
`form`	in	bias_corrected or classic Chao1	—
`chao1`	out	estimated species richness	—
`s_obs`	out	observed number of species	—
`singletons`	out	species seen once (F1)	—
`doubletons`	out	species seen twice (F2)	—

Applies when: whole counts; the classic form needs F1 > 0 and F2 > 0, otherwise the bias-corrected form is used (as scikit-bio does).

```
$ computational-biology solve bio_chao1 "counts=[1, 1, 1, 2, 2, 5, 10]" form=classic
chao1 = 9.25
s_obs = 7
singletons = 3
doubletons = 2
  by: Chao1 species richness estimator  (S_obs + F1^2/(2 F2) (classic) or S_obs + F1(F1 - 1)/(2(F2 + 1)) (bias-corrected))
  form = classic
  assumes: whole counts; the classic form needs F1 > 0 and F2 > 0, otherwise the bias-corrected form is used (as scikit-bio does)
```

What the worked example means: This calculation determines the Chao1 estimate of total species richness by extrapolating from observed abundance data. It uses the number of species actually seen, 7, along with the count of singletons, 3, and doubletons, 2. The resulting estimate, 9.25, suggests that there are likely more species present in the community than were directly observed, accounting for undetected rare species.

Faith's phylogenetic diversity — `bio_faith_pd`

Faith's phylogenetic diversity of a set of observed taxa: the total branch length of the rooted tree connecting them to the root.

$$PD = \sum_{b \in T(\text{taxa}) \cup R} \ell_b$$

symbol		meaning	SI unit
`tree`	in	rooted tree in Newick format with branch lengths	—
`taxa`	in	observed taxa (leaf names separated by spaces)	—
`pd`	out	Faith's PD (sum of branch lengths)	—

Applies when: every branch has a length; taxa are leaves of the tree; rooted convention (path to the root included).

```
$ computational-biology solve bio_faith_pd "tree=((A:1,B:2):3,(C:4,D:5):6);" "taxa=A B"
pd = 6
  by: Faith's phylogenetic diversity  (sum the lengths of every branch on a path from an observed taxon to the root, each once)
  tree = ((A:1,B:2):3,(C:4,D:5):6);
  taxa = A B
  assumes: every branch has a length; taxa are leaves of the tree; rooted convention (path to the root included)
```

What the worked example means: This calculation determines the evolutionary breadth of a specific set of organisms by summing the lengths of all branches in their rooted phylogenetic tree that connect any observed taxon to the root. Using the provided tree structure and list of taxa, the model computes a single value, 6, which quantifies the total amount of unique evolutionary history represented by these species. This metric helps distinguish between sets of species that may have similar counts but vastly different genetic divergence.

Good's coverage of a sample — `bio_goods_coverage`

Good's coverage estimator: the estimated fraction of the community's individuals belonging to species already seen in the sample.

$$C = 1 - \frac{F_1}{N}$$

symbol		meaning	SI unit
`counts`	in	number of individuals of each species	—
`coverage`	out	Good's coverage 1 - F1/N	—

Applies when: whole counts.

```
$ computational-biology solve bio_goods_coverage "counts=[1, 1, 2, 6]"
coverage = 0.8
  by: Good's coverage of a sample  (one minus the proportion of individuals that are singletons)
  assumes: whole counts
```

What the worked example means: The calculation determines Good's coverage by subtracting the proportion of singleton species from one. Using the input counts, the result is 0.8. This value estimates the fraction of the community's individuals belonging to species already observed in the sample, serving as a measure of sampling completeness.

Margalef and Menhinick richness indices — `bio_richness_indices`

Margalef's richness index (S - 1)/ln N and Menhinick's richness index S/sqrt(N) of a sample.

$$D_{Mg} = \frac{S - 1}{\ln N}, \quad D_{Mn} = \frac{S}{\sqrt{N}}$$

symbol		meaning	SI unit
`counts`	in	number of individuals of each species	—
`margalef`	out	Margalef's index (S - 1)/ln N	—
`menhinick`	out	Menhinick's index S/sqrt(N)	—

Applies when: whole counts; N >= 2.

```
$ computational-biology solve bio_richness_indices "counts=[10, 20, 30, 40]"
margalef = 0.6514417229
menhinick = 0.4
  by: Margalef and Menhinick richness indices  (species and individuals counted)
  assumes: whole counts; N >= 2
```

What the worked example means: This calculation determines species richness using two standard indices. The Margalef index, resulting in 0.651442, divides the number of species minus one by the natural logarithm of the total individuals. The Menhinick index, resulting in 0.4, divides the number of species by the square root of the total individuals. These values allow for comparing diversity across samples with different sizes.

Mark-recapture population estimate (Petersen MLE) — `bio_petersen_estimate`

The maximum-likelihood population size from one mark-recapture experiment: M animals marked, C caught later, R of them marked (hypergeometric likelihood).

$$\hat{N} = \left\lfloor \frac{MC}{R} \right\rfloor$$

symbol		meaning	SI unit
`M`	in	number marked and released	—
`C`	in	number caught in the second sample	—
`R`	in	number of marked animals recaptured	—
`N_mle`	out	maximum-likelihood population size (largest maximiser)	—

Applies when: closed population; R >= 1, R <= min(M, C); when MC/R is whole, N = MC/R and MC/R - 1 tie; the larger is given.

```
$ computational-biology solve bio_petersen_estimate M=100 C=80 R=12
N_mle = 666
  by: Mark-recapture population estimate (Petersen MLE)  (the hypergeometric likelihood increases while N <= MC/R: the maximiser
is floor(MC/R))
  assumes: closed population; R >= 1, R <= min(M, C); when MC/R is whole, N = MC/R and MC/R - 1 tie; the larger is given
```

What the worked example means: This calculation determines the maximum likelihood estimate for total population size using single-sample mark-recapture data. Given that 100 animals were initially marked and released, a subsequent sample of 80 animals was captured, of which 12 were found to be marked. The resulting estimate of 666 represents the most probable total number of individuals in the closed population based on the hypergeometric distribution of these counts.

Rarefied species richness — `bio_rarefaction`

The expected number of species in a random subsample of n individuals drawn without replacement (Hurlbert-Sanders rarefaction).

$$E[S_n] = \sum_i \left[1 - \binom{N - N_i}{n} \Big/ \binom{N}{n} \right]$$

symbol		meaning	SI unit
`counts`	in	number of individuals of each species	—
`n`	in	subsample size (individuals)	—
`expected_species`	out	expected number of species in the subsample	—

Applies when: whole counts; 1 <= n <= N.

```
$ computational-biology solve bio_rarefaction "counts=[2, 1, 1]" n=2
expected_species = 1.833333333
  by: Rarefied species richness  (one minus the chance that a species is missed, summed (exact))
  assumes: whole counts; 1 <= n <= N
```

What the worked example means: This calculation determines the expected number of species, 1.83333, when drawing a random subsample of size 2 from the original community. The input counts represent the abundance of each species in the full dataset. By applying exact rarefaction, the model accounts for sampling without replacement to provide a standardized measure of diversity that allows for fair comparison across samples with different total abundances.

Schumacher-Eschmeyer multiple-census estimate — `bio_schumacher_eschmeyer`

The Schumacher-Eschmeyer estimate of a closed population's size from repeated mark-recapture samples.

$$\hat{N} = \frac{\sum_t C_t M_t^2}{\sum_t R_t M_t}$$

symbol		meaning	SI unit
`C`	in	number caught on each occasion	—
`R`	in	number of those already marked	—
`M`	in	number of marked animals at large before each occasion	—
`N`	out	estimated population size	—

Applies when: closed population; whole counts, R <= C; at least one recapture of a marked animal.

```
$ computational-biology solve bio_schumacher_eschmeyer "C=[10, 10, 10]" "R=[0, 2, 3]" "M=[0, 10, 18]"
N = 57.2972973
  by: Schumacher-Eschmeyer multiple-census estimate  (weighted regression of R/C on M through the origin)
  assumes: closed population; whole counts, R <= C; at least one recapture of a marked animal
```

What the worked example means: This calculation determines the total population size using the Schumacher-Eschmeyer method, which applies a weighted linear regression through the origin to relate the number of recaptured marked individuals to the number of marked animals available at large. The result, 57.2973, represents the best estimate of the closed population's size based on the observed capture history where [10, 10, 10] were caught and [0, 2, 3] were already marked during each occasion.

Shannon diversity index — `bio_shannon_diversity`

The Shannon diversity index H' of a community in the stated logarithm base, Pielou's evenness J' and the effective number of species exp(H').

$$H' = -\sum_i p_i \log_b p_i, \quad J' = \frac{H'}{\log_b S}$$

symbol		meaning	SI unit
`counts`	in	number of individuals of each species	—
`base`	in	logarithm base: e (nats), 2 (bits) or 10	—
`H`	out	Shannon index H' in the stated base	—
`evenness`	out	Pielou's evenness J' = H'/ln S	—
`effective_number`	out	effective number of species exp(H') (Hill number of order 1)	—

Applies when: whole non-negative counts; zero counts are not species of the sample.

```
$ computational-biology solve bio_shannon_diversity "counts=[10, 10, 10, 10]" base=e
H = 1.386294361
evenness = 1
effective_number = 4
  by: Shannon diversity index  (H' = -sum p_i log p_i with p_i = n_i / N)
  base = e
  assumes: whole non-negative counts; zero counts are not species of the sample
```

What the worked example means: This calculation determines the biodiversity of a community using the provided counts for each species. The Shannon diversity index, reported as 1.38629, measures uncertainty in predicting the species identity of an individual taken at random from the dataset. From this, Pielou's evenness, shown as 1, indicates how evenly individuals are distributed among the species present. Finally, the effective number of species, given by 4, represents the equivalent number of equally abundant species that would yield the same level of diversity.

Simpson diversity indices — `bio_simpson_diversity`

Simpson's dominance D, Simpson's diversity 1 - D and the inverse Simpson index 1/D, with the stated estimator (infinite population: sum p^2; finite sample: sum n(n-1)/(N(N-1))).

$$D = \sum_i p_i^2 \quad \text{or} \quad D = \frac{\sum n_i(n_i-1)}{N(N-1)}$$

symbol		meaning	SI unit
`counts`	in	number of individuals of each species	—
`estimator`	in	infinite (sum p^2) or finite-sample correction	—
`D`	out	Simpson's dominance D	—
`simpson`	out	Simpson's diversity 1 - D	—
`inverse`	out	inverse Simpson 1/D	—

Applies when: whole non-negative counts; estimator stated.

```
$ computational-biology solve bio_simpson_diversity "counts=[10, 10, 10, 10]" estimator=infinite
D = 0.25
simpson = 0.75
inverse = 4
  by: Simpson diversity indices  (D = sum p_i^2 or sum n_i(n_i - 1)/(N(N - 1)))
  estimator = infinite
  assumes: whole non-negative counts; estimator stated
```

What the worked example means: This calculation determines the Simpson's dominance index, which measures the probability that two individuals randomly selected from a sample belong to the same species. Using the provided counts for each species and the specified estimator method, the model computes the dominance value 0.25. From this, it derives the Simpson's diversity index as one minus the dominance, resulting in 0.75, and the inverse Simpson index, representing the effective number of species, calculated as 4. These metrics collectively describe how evenly individuals are distributed among the observed species.

Species-area relationship fit — `bio_species_area`

The constants c and z of the Arrhenius species-area relationship S = c A^z fitted by least squares on log S against log A.

$$\ln S = \ln c + z \ln A$$

symbol		meaning	SI unit
`A`	in	areas (any one unit)	—
`S`	in	species counts	—
`c`	out	constant c (species in a unit area)	—
`z`	out	exponent z	—
`r2`	out	r^2 of the log-log fit	—

Applies when: positive areas and counts; c refers to the area unit used.

```
$ computational-biology solve bio_species_area "A=[1, 10, 100, 1000]" "S=[10, 20, 40, 80]"
c = 10
z = 0.3010299957
r2 = 1
  by: Species-area relationship fit  (ordinary least squares of ln S on ln A: slope z, intercept ln c)
  assumes: positive areas and counts; c refers to the area unit used
```

What the worked example means: This calculation determines the parameters of the Arrhenius species-area relationship by applying ordinary least squares to the logarithm of species counts against the logarithm of areas. The resulting constant c, valued at 10, represents the expected number of species in a unit area, while the exponent z, valued at 0.30103, describes the rate at which species richness increases with area. The coefficient of determination r-squared is 1, indicating how well the power-law model explains the variation in the data.

Enzyme kinetics (9)

Competitive inhibition global fit — `bio_competitive_inhibition_fit`

Vmax, Km and the inhibition constant Ki of a competitive inhibitor fitted globally to rates measured at several substrate and inhibitor concentrations (nonlinear least squares, unweighted).

$$v = \frac{V_{\max} S}{K_m \left(1 + \frac{[I]}{K_i}\right) + S}$$

symbol		meaning	SI unit
`S`	in	substrate concentrations (one per measurement)	—
`I`	in	inhibitor concentrations (one per measurement, 0 for none)	—
`v`	in	measured rates	—
`Vmax`	out	maximum rate	—
`Km`	out	Michaelis constant (unit of S)	—
`Ki`	out	inhibition constant (unit of I)	—

Applies when: rates without inhibitor at 3 or more S; rates with inhibitor; convergence.

```
$ computational-biology solve bio_competitive_inhibition_fit "S=[1, 2, 4, 8, 1, 2, 4, 8]" "I=[0, 0, 0, 0, 2, 2, 2, 2]" "v=[5,
6.66667, 8, 8.88889, 2.5, 4, 5.71429, 7.27273]"
Vmax = 10.00000204
Km = 1.000000099
Ki = 1.000000711
  by: Competitive inhibition global fit  (Levenberg-Marquardt on v = Vmax S / (Km (1 + I/Ki) + S), started from the uninhibited
fit)
  assumes: rates without inhibitor at 3 or more S; rates with inhibitor; convergence
```

What the worked example means: This calculation determines the kinetic parameters for competitive inhibition by fitting the observed reaction rates to the theoretical model. The result provides the maximum velocity, which is 10, alongside the substrate affinity constant, 1. It

also estimates the inhibitor dissociation constant, 1, indicating how strongly the inhibitor binds to the enzyme relative to the substrate.

Eadie-Hofstee fit of Michaelis-Menten data — `bio_eadie_hofstee`

Vmax and Km from a least-squares line through the Eadie-Hofstee plot of v against v/S.

$$v = V_{\max} - K_m \frac{v}{S}$$

symbol		meaning	SI unit
`S`	in	substrate concentrations (any one unit)	—
`v`	in	initial rates (any one unit)	—
`Vmax`	out	maximum rate (unit of v)	—
`Km`	out	Michaelis constant (unit of S)	—
`r2`	out	coefficient of determination of the linear fit	—

Applies when: positive S and v; negative slope, positive intercept.

```
$ computational-biology solve bio_eadie_hofstee "S=[1, 2, 4, 8]" "v=[5, 6.66667, 8, 8.88889]"
Vmax = 10.0000015
Km = 0.9999999798
r2 = 1
  by: Eadie-Hofstee fit of Michaelis-Menten data  (ordinary least squares of v on v/S: intercept Vmax, slope -Km)
  assumes: positive S and v; negative slope, positive intercept
```

What the worked example means: This calculation performs an ordinary least squares regression on the Eadie-Hofstee plot, where reaction velocity is plotted against the ratio of velocity to substrate concentration. The y-intercept of this linear fit provides the estimate for the maximum reaction rate, which is 10, while the negative slope gives the Michaelis constant, estimated as 1. The goodness of fit is quantified by the coefficient of determination, reported here as 1, indicating how well the linear model explains the variance in the observed data.

Hanes-Woolf fit of Michaelis-Menten data — `bio_hanes_woolf`

Vmax and Km from a least-squares line through the Hanes-Woolf plot of S/v against S.

$$\frac{S}{v} = \frac{1}{V_{\max}} S + \frac{K_m}{V_{\max}}$$

symbol		meaning	SI unit
`S`	in	substrate concentrations (any one unit)	—
`v`	in	initial rates (any one unit)	—
`Vmax`	out	maximum rate (unit of v)	—
`Km`	out	Michaelis constant (unit of S)	—
`r2`	out	coefficient of determination of the linear fit	—

Applies when: positive S and v; positive slope and intercept.

```
$ computational-biology solve bio_hanes_woolf "S=[1, 2, 4, 8]" "v=[5, 6.66667, 8, 8.88889]"
Vmax = 10.00000075
Km = 0.9999997
r2 = 1
  by: Hanes-Woolf fit of Michaelis-Menten data  (ordinary least squares of S/v on S: slope 1/Vmax, intercept Km/Vmax)
  assumes: positive S and v; positive slope and intercept
```

What the worked example means: This calculation determines the kinetic parameters for an enzyme reaction by performing ordinary least squares regression on the Hanes-Woolf plot, which graphs substrate concentration divided by initial velocity against substrate concentration. The slope of this linear fit corresponds to the reciprocal of the maximum rate, while the intercept represents the ratio of the Michaelis constant to the maximum rate. Using the input data, the model computes a maximum rate of 10 and a Michaelis constant of 1. The quality of this linear approximation is indicated by the coefficient of determination, calculated here as 1, showing how well the transformed data aligns with the theoretical linear relationship.

Hill coefficient from a Hill plot — `bio_hill_plot`

The Hill coefficient n and half-saturation constant K from a least-squares line through the Hill plot of log10(Y/(1-Y)) against log10 S (Y = fractional saturation).

$$\log \frac{Y}{1-Y} = n \log S - n \log K$$

symbol		meaning	SI unit
`S`	in	ligand or substrate concentrations	—
`Y`	in	fractional saturations (0 < Y < 1)	—
`n`	out	Hill coefficient	—

symbol		meaning	SI unit
`K`	out	half-saturation concentration (unit of S)	—
`r2`	out	coefficient of determination of the line	—

Applies when: 0 < Y < 1; positive S; rising Hill plot.

```
$ computational-biology solve bio_hill_plot "S=[1, 2, 4, 8]" "Y=[0.0588235, 0.2, 0.5, 0.8]"
n = 2.00000023
K = 4.000000106
r2 = 1
  by: Hill coefficient from a Hill plot  (ordinary least squares of log10(Y/(1-Y)) on log10 S: slope n, intercept -n log10 K)
  assumes: 0 < Y < 1; positive S; rising Hill plot
```

What the worked example means: This calculation determines the cooperativity and binding affinity from saturation data. By plotting the logarithm of the ratio of bound to unbound fraction against the logarithm of concentration, a straight line is fitted using least squares. The slope of this line yields the Hill coefficient 2, indicating how sharply saturation changes with concentration, while the intercept relates to the half-saturation constant 4, representing the concentration at which half the sites are occupied. The coefficient of determination 1 quantifies how well the linear model fits the transformed data.

Hill equation nonlinear least-squares fit — `bio_hill_fit`

Vmax, the half-saturation constant K and the Hill coefficient n fitted to rate or binding data by nonlinear least squares (unweighted).

$$v = \frac{V_{\max} S^n}{K^n + S^n}$$

symbol		meaning	SI unit
`S`	in	concentrations (any one unit)	—
`v`	in	rates or responses (any one unit)	—
`Vmax`	out	maximum rate	—
`K`	out	half-saturation concentration (unit of S)	—
`n`	out	Hill coefficient	—

Applies when: at least 4 distinct positive concentrations; positive responses; convergence.

```
$ computational-biology solve bio_hill_fit "S=[1, 2, 4, 8, 16]" "v=[0.588235, 2, 5, 8, 9.41176]"
Vmax = 9.999993248
K = 3.999996626
n = 2.000001561
  by: Hill equation nonlinear least-squares fit  (Levenberg-Marquardt on v = Vmax S^n / (K^n + S^n), started from the Hill-plot
line)
  assumes: at least 4 distinct positive concentrations; positive responses; convergence
```

What the worked example means: This calculation determines the parameters that best describe the relationship between concentration and response using the Hill equation. It estimates the maximum possible rate, which is 10, and the concentration required to reach half of that maximum, known as the half-saturation constant or K, which is 4. Additionally, it quantifies cooperativity through the Hill coefficient, n, calculated here as 2. These values define the sigmoidal curve fitted to the input data points.

Lineweaver-Burk fit of Michaelis-Menten data — `bio_lineweaver_burk`

Vmax and Km from a least-squares line through the double-reciprocal (Lineweaver-Burk) plot of 1/v against 1/S.

$$\frac{1}{v} = \frac{K_m}{V_{\max}} \frac{1}{S} + \frac{1}{V_{\max}}$$

symbol		meaning	SI unit
`S`	in	substrate concentrations (any one unit)	—
`v`	in	initial rates (any one unit)	—
`Vmax`	out	maximum rate (unit of v)	—
`Km`	out	Michaelis constant (unit of S)	—
`r2`	out	coefficient of determination of the linear fit	—

Applies when: positive S and v; positive slope and intercept.

```
$ computational-biology solve bio_lineweaver_burk "S=[1, 2, 4, 8]" "v=[5, 6.66667, 8, 8.88889]"
Vmax = 10.00000249
Km = 1.000000304
r2 = 1
  by: Lineweaver-Burk fit of Michaelis-Menten data  (ordinary least squares of 1/v on 1/S: intercept 1/Vmax, slope Km/Vmax)
  assumes: positive S and v; positive slope and intercept
```

What the worked example means: This calculation determines the kinetic parameters for an enzyme reaction by applying ordinary least squares to the double reciprocal plot. The input substrate concentrations and initial rates are transformed into their reciprocals, allowing a

linear regression to estimate the y-intercept as the inverse of the maximum velocity and the slope as the ratio of the Michaelis constant to the maximum velocity. The resulting maximum rate is 10 and the Michaelis constant is 1, indicating how quickly the enzyme reaches saturation and its affinity for the substrate respectively.

Michaelis-Menten nonlinear least-squares fit — `bio_michaelis_menten_fit`

Vmax and Km of the Michaelis-Menten equation fitted to rate data by nonlinear least squares (unweighted).

$$\min_{V_{\max},K_m} \sum_i \left(v_i - \frac{V_{\max} S_i}{K_m + S_i}\right)^2$$

symbol		meaning	SI unit
`S`	in	substrate concentrations (any one unit)	—
`v`	in	initial rates (any one unit)	—
`Vmax`	out	maximum rate (unit of v)	—
`Km`	out	Michaelis constant (unit of S)	—

Applies when: at least 3 distinct positive S; positive v; an interior least-squares optimum.

```
$ computational-biology solve bio_michaelis_menten_fit "S=[1, 2, 4, 8]" "v=[5, 6.66667, 8, 8.88889]"
Vmax = 10.00000077
Km = 0.999999678
  by: Michaelis-Menten nonlinear least-squares fit  (variable projection: Vmax is linear given Km; the best Km zeroes the
derivative of the projected fit)
  assumes: at least 3 distinct positive S; positive v; an interior least-squares optimum
```

What the worked example means: This analysis computes the enzyme kinetic parameters Vmax and Km by fitting the Michaelis-Menten equation to measured initial rates. The resulting maximum velocity is 10, representing the rate at saturation, while the Michaelis constant is 1, indicating the substrate concentration needed to reach half of that maximum speed.

Substrate remaining in a Michaelis-Menten reaction — `bio_michaelis_menten_progress`

The substrate and product concentrations after a time t of an irreversible Michaelis-Menten reaction (integrated rate law; constant enzyme, no product inhibition).

$$K_m \ln\frac{S_0}{S} + (S_0 - S) = V_{\max} t$$

symbol		meaning	SI unit
`S0`	in	initial substrate concentration	—
`Vmax`	in	maximum rate (concentration per time)	—
`Km`	in	Michaelis constant (concentration)	—
`t`	in	elapsed time (time unit of Vmax)	—
`S`	out	substrate concentration at t	—
`P`	out	product concentration at t	—

Applies when: positive S0, Vmax, Km; t >= 0; consistent units.

```
$ computational-biology solve bio_michaelis_menten_progress S0=10 Vmax=1 Km=2 t=5
S = 6.016243923
P = 3.983756077
  by: Substrate remaining in a Michaelis-Menten reaction  (solve Km ln(S0/S) + (S0 - S) = Vmax t by bisection (for P below half
conversion, for ln S above))
  S0 = 10
  Vmax = 1
  Km = 2
  t = 5
  assumes: positive S0, Vmax, Km; t >= 0; consistent units
```

What the worked example means: This calculation determines the remaining substrate and formed product after a specified time in an irreversible Michaelis-Menten reaction. It solves the integrated rate law equation Km ln(S0/S) + (S0 - S) = Vmax t using bisection to find the substrate concentration at time t. The result shows that with initial substrate 10, maximum rate 1, and Michaelis constant 2, the system evolves such that substrate decreases while product increases, providing a complete progress curve rather than just an initial rate.

Time to reach a conversion in a Michaelis-Menten reaction — `bio_michaelis_menten_time`

The time an irreversible Michaelis-Menten reaction needs to convert a stated fraction of its substrate.

$$t = \frac{K_m \ln\frac{1}{1-X} + S_0 X}{V_{\max}}$$

symbol		meaning	SI unit
`S0`	in	initial substrate concentration	—
`Vmax`	in	maximum rate (concentration per time)	—
`Km`	in	Michaelis constant (concentration)	—
`X`	in	fraction of substrate converted (0 to 1, or a percentage)	—
`t`	out	time (time unit of Vmax)	—

Applies when: 0 < X < 1; positive S0, Vmax, Km.

```
$ computational-biology solve bio_michaelis_menten_time S0=10 Vmax=1 Km=2 X=0.5
t = 6.386294361
  by: Time to reach a conversion in a Michaelis-Menten reaction  (integrated Michaelis-Menten equation)
  S0 = 10
  Vmax = 1
  Km = 2
  X = 0.5
  assumes: 0 < X < 1; positive S0, Vmax, Km
```

What the worked example means: This calculation determines the duration required for an irreversible Michaelis-Menten reaction to achieve a specific conversion level. Using the integrated rate law, it accounts for the decreasing substrate concentration as the reaction proceeds. With an initial substrate of 10, maximum velocity of 1, and Michaelis constant of 2, reaching a conversion fraction of 0.5 takes exactly 6.38629. This result helps predict when a biochemical assay or metabolic pathway will reach a desired state.

Genetics (5)

Inbreeding coefficient from a pedigree — `bio_pedigree_inbreeding`

Wright's inbreeding coefficient F of an individual from its pedigree (founders unrelated and not inbred).

$$F_X = \sum_{\text{paths}} \left(\tfrac{1}{2}\right)^{n_1+n_2+1}(1+F_A)$$

symbol		meaning	SI unit
`pedigree`	in	parent-child pairs, e.g. [P-S, Q-S, P-D, Q-D, S-X, D-X]	—
`individual`	in	name of the individual	—
`F`	out	inbreeding coefficient	—

Applies when: every non-founder has exactly two parents listed; no cycles; founders unrelated, F = 0.

```
$ computational-biology solve bio_pedigree_inbreeding "pedigree=P-S, Q-S, P-D, Q-D, S-X, D-X" individual=X
F = 0.25
  by: Inbreeding coefficient from a pedigree  (Wright's path counting: sum over common ancestors and non-overlapping paths of
(1/2)^(n1+n2+1) (1 + F_A))
  individual = X
  assumes: every non-founder has exactly two parents listed; no cycles; founders unrelated, F = 0
```

What the worked example means: This calculation determines the inbreeding coefficient, denoted as F, for a specific individual based on their pedigree. The value 0.25 represents the probability that two alleles at any given locus are identical by descent from a common ancestor. This metric is derived using Wright's path counting method, which sums contributions from all non-overlapping paths connecting the parents through their shared ancestors. A higher value indicates greater genetic similarity between the parents and increased risk of homozygosity for deleterious recessive traits.

Map distance from a recombination fraction — `bio_map_distance`

The genetic map distance (cM) corresponding to a recombination fraction, by the Haldane or Kosambi map function.

$$d_H = -\tfrac{1}{2}\ln(1-2r), \quad d_K = \tfrac{1}{4}\ln\frac{1+2r}{1-2r}$$

symbol		meaning	SI unit
`r`	in	recombination fraction (0 to 0.5)	—
`map`	in	map function: haldane (no interference) or kosambi	—
`d_cM`	out	map distance in centimorgans	—

Applies when: 0 <= r < 0.5; map function stated.

```
$ computational-biology solve bio_map_distance r=0.2 map=haldane
d_cM = 25.54128119
  by: Map distance from a recombination fraction  (Haldane d = -ln(1-2r)/2; Kosambi d = ln((1+2r)/(1-2r))/4 (Morgans, x100 for
cM))
  r = 0.2
  map = haldane
  assumes: 0 <= r < 0.5; map function stated
```

What the worked example means: This calculation converts an observed recombination fraction into a genetic map distance measured in centimorgans. By applying the selected map function, specifically haldane, to the input value of 0.2, the tool accounts for crossover interference to provide a more accurate estimate of physical distance than raw recombination data alone. The resulting map distance is 25.5413 centimorgans.

Phenotype ratio of a cross (complete dominance) — `bio_cross_phenotype_ratio`

The offspring phenotype ratio of a cross with complete dominance, the probabilities of the all-dominant and all-recessive phenotypes and the numbers of genotypes and phenotypes.

$$(\tfrac{3}{4}D + \tfrac{1}{4}r)^n \Rightarrow 9:3:3:1 \text{ for two loci}$$

symbol		meaning	SI unit
`parent1`	in	genotype of the first parent, letter pairs (e.g. AaBb)	—
`parent2`	in	genotype of the second parent	—
`p_all_dominant`	out	probability of showing every dominant trait	—
`p_all_recessive`	out	probability of showing every recessive trait	—
`n_genotypes`	out	number of distinct offspring genotypes	—
`n_phenotypes`	out	number of distinct offspring phenotypes	—
`ratio`	out	phenotype ratio, largest class first (e.g. 9:3:3:1)	—

Applies when: complete dominance at every locus; independent assortment.

```
$ computational-biology solve bio_cross_phenotype_ratio parent1=AaBb parent2=AaBb
p_all_dominant = 0.5625
p_all_recessive = 0.0625
n_genotypes = 9
n_phenotypes = 4
ratio = 9:3:3:1
  by: Phenotype ratio of a cross (complete dominance)  (per-locus phenotype probabilities (3/4 : 1/4 for Aa x Aa) multiplied
across loci)
  parent1 = AaBb
  parent2 = AaBb
  assumes: complete dominance at every locus; independent assortment
```

What the worked example means: This calculation determines the expected offspring outcomes for a genetic cross involving complete dominance and independent assortment. Analyzing the parental genotypes AaBb and AaBb gives the specific phenotype ratio 9:3:3:1 and the count of distinct phenotypes 4. The result also includes the probability that an offspring exhibits every dominant trait, which is 0.5625, and the probability of showing every recessive trait, given as 0.0625. Additionally, the total number of unique genotypes produced by this cross is 9. These values describe the statistical distribution of traits in the next generation under Mendelian assumptions.

Probability of an offspring genotype in a cross — `bio_cross_genotype_probability`

The probability that an offspring of two parents has a stated genotype (monohybrid, dihybrid or more loci; independent assortment).

$$P = \prod_{\ell} P_\ell(g_\ell)$$

symbol		meaning	SI unit
`parent1`	in	genotype of the first parent, letter pairs (e.g. AaBb)	—
`parent2`	in	genotype of the second parent	—
`offspring`	in	offspring genotype asked about	—
`probability`	out	probability of the offspring genotype	—
`fraction`	out	the same as a fraction	—

Applies when: the same loci in the same order; independent assortment; at most 6 loci.

```
$ computational-biology solve bio_cross_genotype_probability parent1=Aa parent2=Aa offspring=Aa
probability = 0.5
fraction = 1/2
  by: Probability of an offspring genotype in a cross  (product over loci of the Punnett-square probability of each locus
genotype)
  offspring = Aa
  parent2 = Aa
  parent1 = Aa
  assumes: the same loci in the same order; independent assortment; at most 6 loci
```

What the worked example means: This calculation determines the likelihood that an offspring from parents with genotypes Aa and Aa will have the specific genotype Aa. It works by finding the probability for each gene locus independently using Punnett squares and then multiplying those probabilities together. The final result is a probability value of 0.5, which can also be expressed as the fraction 1/2. This method assumes that the genes are located on different chromosomes or are far enough apart to assort independently.

Recombination fraction from a map distance — `bio_recombination_fraction`

The recombination fraction between two loci a stated map distance apart, by the Haldane or Kosambi map function.

$$r_H = \tfrac{1}{2}(1 - e^{-2d}), \quad r_K = \tfrac{1}{2}\tanh 2d$$

symbol		meaning	SI unit
`d_cM`	in	map distance in centimorgans	—
`map`	in	map function: haldane (no interference) or kosambi	—
`r`	out	recombination fraction	—

Applies when: d >= 0; map function stated.

```
$ computational-biology solve bio_recombination_fraction d_cM=50 map=haldane
r = 0.3160602794
  by: Recombination fraction from a map distance  (Haldane r = (1 - e^{-2d})/2; Kosambi r = tanh(2d)/2 (d in Morgans))
  d_cM = 50
  map = haldane
  assumes: d >= 0; map function stated
```

What the worked example means: This calculation determines the probability of recombination between two genetic loci based on their stated map distance. By applying the selected map function to the input distance of 50, the model computes a recombination fraction of 0.31606. This value represents the likelihood that crossing over will separate these loci during meiosis, providing a direct measure of linkage strength.

Microbial growth (1)

Chemostat steady state (Monod kinetics) — `bio_chemostat_steady_state`

The steady-state substrate and biomass concentrations of a chemostat with Monod growth kinetics.

$$\bar{S} = \frac{K_s D}{\mu_{\max} - D}, \quad \bar{X} = Y(S_0 - \bar{S})$$

symbol		meaning	SI unit
`mu_max`	in	maximum specific growth rate (1/time)	—
`Ks`	in	half-saturation constant (concentration)	—
`Y`	in	biomass yield (biomass per substrate)	—
`D`	in	dilution rate (1/time, same unit as mu_max)	—
`S0`	in	feed substrate concentration	—
`S`	out	steady-state substrate concentration	—
`X`	out	steady-state biomass concentration	—

Applies when: D < mu_max; no washout (S < S0); consistent units.

```
$ computational-biology solve bio_chemostat_steady_state mu_max=1 Ks=1 Y=0.5 D=0.5 S0=10
S = 1
X = 4.5
  by: Chemostat steady state (Monod kinetics)  (growth balances dilution: mu(S) = D gives S; the substrate balance gives X)
  mu_max = 1
  Ks = 1
  Y = 0.5
  D = 0.5
  S0 = 10
  assumes: D < mu_max; no washout (S < S0); consistent units
```

What the worked example means: This calculation determines the equilibrium concentrations of substrate and biomass in a chemostat operating under Monod kinetics. Balancing the specific growth rate against the dilution rate shows that the residual substrate level settles at 1. The corresponding biomass density, derived from the yield coefficient and the consumed substrate, reaches a stable value of 4.5. These results indicate how efficiently the culture converts feed into cell mass under the specified flow conditions.

Phylogenetics (7)

Jukes-Cantor distance — `bio_jukes_cantor_distance`

The Jukes-Cantor (JC69) evolutionary distance between two aligned DNA sequences: the expected number of substitutions per site, corrected for multiple hits.

$$d = -\tfrac{3}{4}\ln\left(1 - \tfrac{4}{3}p\right)$$

symbol		meaning	SI unit
`seq1`	in	first aligned DNA sequence	—
`seq2`	in	second aligned DNA sequence	—

symbol		meaning	SI unit
`p`	out	proportion of differing sites	—
`d`	out	Jukes-Cantor distance (substitutions per site)	—

Applies when: equal lengths, no gaps; p < 0.75.

```
$ computational-biology solve bio_jukes_cantor_distance seq1=AAAAAAAAAA seq2=AAAAAAAACC
p = 0.2
d = 0.2326161962
  by: Jukes-Cantor distance  (d = -(3/4) ln(1 - 4p/3))
  seq2 = AAAAAAAACC
  seq1 = AAAAAAAAAA
  assumes: equal lengths, no gaps; p < 0.75
```

What the worked example means: This calculation determines the evolutionary distance between two DNA sequences by correcting for multiple hits. The input sequences were compared to find that the proportion of differing sites is 0.2. Applying the correction formula yields a final distance of 0.232616, which represents the estimated number of substitutions per site since the sequences diverged.

Kimura two-parameter distance — `bio_kimura_distance`

The Kimura two-parameter (K80) distance between two aligned DNA sequences, separating transitions (A<->G, C<->T) from transversions.

$$d = -\tfrac{1}{2}\ln(1 - 2P - Q) - \tfrac{1}{4}\ln(1 - 2Q)$$

symbol		meaning	SI unit
`seq1`	in	first aligned DNA sequence	—
`seq2`	in	second aligned DNA sequence	—
`d`	out	K80 distance (substitutions per site)	—

Applies when: equal lengths, no gaps; 1 - 2P - Q > 0 and 1 - 2Q > 0.

```
$ computational-biology solve bio_kimura_distance seq1=AAAAAAAAAA seq2=AAAAAAAGGC
d = 0.4023594781
  by: Kimura two-parameter distance  (P = transition proportion, Q = transversion proportion, d = -ln(1-2P-Q)/2 - ln(1-2Q)/4)
  seq1 = AAAAAAAAAA
  seq2 = AAAAAAAGGC
  assumes: equal lengths, no gaps; 1 - 2P - Q > 0 and 1 - 2Q > 0
```

What the worked example means: This calculation determines the evolutionary distance between two aligned DNA sequences using the Kimura two-parameter model. It accounts for different rates of transitions and transversions by analyzing the proportions of these changes in the input sequences. The resulting value, 0.402359, represents the estimated number of nucleotide substitutions per site that have occurred since the sequences diverged from a common ancestor.

Neighbour-joining branch lengths — `bio_neighbor_joining`

The total tree length and the terminal branch lengths of the neighbour-joining tree of a distance matrix.

$$Q_{ij} = (r-2)d_{ij} - R_i - R_j, \quad \ell_i = \tfrac{1}{2}d_{ij} + \frac{R_i - R_j}{2(r-2)}$$

symbol		meaning	SI unit
`D`	in	symmetric distance matrix, zero diagonal (at least 3 taxa)	—
`total_length`	out	sum of all branch lengths	—
`branch`	out	terminal branch length of each taxon, in input order	—

Applies when: at least 3 taxa; no ties in the joining criterion; negative branch lengths kept as computed.

```
$ computational-biology solve bio_neighbor_joining "D=[[0, 5, 9, 9, 8], [5, 0, 10, 10, 9], [9, 10, 0, 8, 7], [9, 10, 8, 0, 3], [8,
9, 7, 3, 0]]"
total_length = 17
branch = [2.0,3.0,4.0,2.0,1.0]
  by: Neighbour-joining branch lengths  (Saitou-Nei: join the pair minimising Q = (r-2) d_ij - R_i - R_j; branch lengths from the
row sums)
  assumes: at least 3 taxa; no ties in the joining criterion; negative branch lengths kept as computed
```

What the worked example means: This calculation determines the structure of a phylogenetic tree using the Saitou-Nei neighbour-joining algorithm. It starts with a symmetric distance matrix representing evolutionary distances between taxa. The method iteratively joins pairs that minimize a specific criterion based on their distances to all other taxa, adjusting for the number of remaining nodes. The result is a tree where the sum of all edge weights equals 17, and the individual lengths from each taxon to its nearest internal node are provided in [2, 3, 4, 2, 1]. This approach does not assume equal rates of evolution across lineages, allowing for non-ultrametric trees that reflect variable divergence speeds.

Number of possible phylogenetic trees — `bio_tree_count`

The number of distinct labelled bifurcating trees for n taxa, rooted or unrooted.

$$N_{\text{rooted}} = (2n-3)!!, \quad N_{\text{unrooted}} = (2n-5)!!$$

symbol		meaning	SI unit
`n`	in	number of taxa	—
`rooted`	in	rooted or unrooted trees	—
`count`	out	number of distinct bifurcating trees	—

Applies when: n >= 2 (rooted) or n >= 3 (unrooted); n <= 100.

```
$ computational-biology solve bio_tree_count n=5 rooted=unrooted
count = 15
  by: Number of possible phylogenetic trees  (double factorials: (2n-3)!! rooted, (2n-5)!! unrooted)
  rooted = unrooted
  assumes: n >= 2 (rooted) or n >= 3 (unrooted); n <= 100
```

What the worked example means: This calculation determines the total count of unique labeled phylogenetic tree structures possible for a specified set of taxa. Using double factorials based on whether the trees are rooted or unrooted, it computes that there are 15 distinct binary tree topologies for the input of 5 taxa.

Parsimony score of a tree (Fitch) — `bio_fitch_parsimony`

The minimum number of character changes (small parsimony score) needed on a given rooted bifurcating tree to explain aligned sequences at its leaves.

$$c_v(a) = \sum_{w \in \text{children}(v)} \min_b \left(c_w(b) + [a \neq b] \right)$$

symbol		meaning	SI unit
`tree`	in	rooted bifurcating tree in Newick format	—
`states`	in	aligned sequence of each leaf, written name:SEQUENCE separated by spaces	—
`score`	out	parsimony score (number of changes)	—

Applies when: strictly bifurcating rooted tree; one aligned sequence per leaf; unit cost per change.

```
$ computational-biology solve bio_fitch_parsimony "tree=((A,B),(C,D));" "states=A:AACG B:AATG C:GACT D:GGCT"
score = 4
  by: Parsimony score of a tree (Fitch)  (dynamic programming over the tree site by site (equivalent to Fitch's algorithm with
unit costs))
  tree = ((A,B),(C,D));
  states = A:AACG B:AATG C:GACT D:GGCT
  assumes: strictly bifurcating rooted tree; one aligned sequence per leaf; unit cost per change
```

What the worked example means: This calculation determines the parsimony score for the provided rooted bifurcating tree and aligned leaf sequences. By applying dynamic programming equivalent to Fitch's algorithm with unit costs, it computes the smallest possible number of character state transitions needed to account for the data at the leaves. The resulting value, 4, represents the total count of mutations or changes inferred across all sites in the alignment.

Robinson-Foulds distance between trees — `bio_robinson_foulds`

The Robinson-Foulds (symmetric-difference) distance between two trees on the same taxa, comparing their bipartitions (trees treated as unrooted), and the distance as a proportion.

$$\mathrm{RF} = |S_1 \setminus S_2| + |S_2 \setminus S_1|$$

symbol		meaning	SI unit
`tree1`	in	first tree in Newick format	—
`tree2`	in	second tree in Newick format	—
`rf`	out	number of bipartitions found in only one tree	—
`rf_normalized`	out	RF divided by the total number of non-trivial bipartitions of both trees	—

Applies when: the same taxa; at least 4 taxa; trees compared unrooted; branch lengths ignored.

```
$ computational-biology solve bio_robinson_foulds "tree1=((A,B),(C,D),(E,F));" "tree2=((A,C),(B,D),(E,F));"
rf = 4
rf_normalized = 0.6666666667
  by: Robinson-Foulds distance between trees  (non-trivial bipartitions of each tree; count those not shared)
  tree1 = ((A,B),(C,D),(E,F));
  tree2 = ((A,C),(B,D),(E,F));
  assumes: the same taxa; at least 4 taxa; trees compared unrooted; branch lengths ignored
```

What the worked example means: This calculation determines the topological difference between two unrooted phylogenetic trees by comparing their sets of non-trivial bipartitions. The result, 4, represents the count of splits found in one tree but not the other, while 0.666667 expresses this difference as a proportion of the total possible distinct splits. A lower value indicates greater similarity in the inferred evolutionary relationships.

UPGMA tree heights — `bio_upgma`

The node heights of the UPGMA (average-linkage) tree of a distance matrix: the root height and every merge height (half the distance between the merged clusters).

$$d_{(ij),k} = \frac{n_i d_{ik} + n_j d_{jk}}{n_i + n_j}, \quad h = d_{ij}/2$$

symbol		meaning	SI unit
`D`	in	symmetric distance matrix, zero diagonal	—
`root_height`	out	height of the root	—
`heights`	out	all node heights, ascending	—

Applies when: symmetric, zero diagonal, positive distances; no ties in a merge step.

```
$ computational-biology solve bio_upgma "D=[[0, 2, 4, 6], [2, 0, 4, 6], [4, 4, 0, 6], [6, 6, 6, 0]]"
root_height = 3
heights = [1.0,2.0,3.0]
  by: UPGMA tree heights  (repeatedly merge the closest clusters; distances to a merged cluster are size-weighted averages)
  assumes: symmetric, zero diagonal, positive distances; no ties in a merge step
```

What the worked example means: This calculation determines the node heights for a hierarchical clustering tree using the Unweighted Pair Group Method with Arithmetic Mean. Starting from the symmetric distance matrix [0 2 4 6; 2 0 4 6; 4 4 0 6; 6 6 6 0], the algorithm repeatedly merges the two closest clusters, calculating new distances as size-weighted averages. Each merge step produces a node height equal to half the distance between the merged clusters. The final result provides the root height 3 and all internal node heights [1, 2, 3], which represent the evolutionary time or divergence levels under the assumption of a constant molecular clock.

Population biology (2)

Leslie matrix growth rate and stable age distribution — `bio_leslie_growth_rate`

The asymptotic growth rate (dominant eigenvalue lambda and r = ln lambda), the net reproductive rate R0 and the stable age distribution of a Leslie model.

$$\sum_i F_i \, l_i \, \lambda^{-i} = 1, \quad R_0 = \sum_i F_i l_i$$

symbol		meaning	SI unit
`F`	in	fecundity of each age class	—
`S`	in	survival from each age class to the next (one fewer than F)	—
`lambda1`	out	dominant eigenvalue (growth factor per step)	—
`r`	out	intrinsic rate ln(lambda)	—
`R0`	out	net reproductive rate (expected offspring per newborn)	—
`stable`	out	stable age distribution (fractions summing to 1)	—

Applies when: 0 < S_i <= 1; F_i >= 0, at least one positive.

```
$ computational-biology solve bio_leslie_growth_rate "F=[0, 2]" "S=[0.5]"
lambda1 = 1
r = 0
R0 = 1
stable = [0.666666666667,0.333333333333]
  by: Leslie matrix growth rate and stable age distribution  (the positive root of the Euler-Lotka equation sum F_i l_i lambda^-i
= 1; stable distribution proportional to l_i lambda^-(i-1))
  assumes: 0 < S_i <= 1; F_i >= 0, at least one positive
```

What the worked example means: This calculation determines the asymptotic growth characteristics of a population using Leslie matrix theory. Solving for the dominant eigenvalue gives the growth factor per time step, which is 1. This value translates to an intrinsic growth rate of 0. The net reproductive rate, representing the expected number of offspring produced by a newborn over its lifetime, is calculated as 1. Finally, the stable age distribution, showing the proportional makeup of each age class once transient dynamics settle, is given by [0.666667, 0.333333]. These outputs rely on the input fecundity values [0, 2] and survival rates [0.5].

Leslie matrix population projection — `bio_leslie_projection`

The age-class numbers and total population after t time steps of an age-structured (Leslie) model.

$$\mathbf{n}_{t+1} = L\,\mathbf{n}_t, \quad L = \begin{pmatrix} F_1 & F_2 & \cdots \\ S_1 & 0 & \\ & \ddots & \end{pmatrix}$$

symbol		meaning	SI unit
`F`	in	fecundity of each age class	—
`S`	in	survival from each age class to the next (one fewer than F)	—
`n0`	in	initial number in each age class	—
`t`	in	number of time steps	—
`n_t`	out	number in each age class after t steps	—
`total`	out	total population after t steps	—

Applies when: 0 < S_i <= 1; F_i >= 0, at least one positive; t <= 500.

```
$ computational-biology solve bio_leslie_projection "F=[0, 1.5, 1]" "S=[0.5, 0.8]" "n0=[100, 50, 20]" t=1
n_t = [95.0,50.0,40.0]
total = 185
  by: Leslie matrix population projection  (multiply the age vector by the Leslie matrix t times)
  assumes: 0 < S_i <= 1; F_i >= 0, at least one positive; t <= 500
```

What the worked example means: This calculation projects the future state of an age-structured population by multiplying the initial age vector by the Leslie matrix repeatedly. The resulting age distribution after 1 steps is given by [95, 50, 40], and the sum of these values yields the total population size of 185. This outcome reflects how specific survival rates and fecundity levels interact over discrete time intervals to shape demographic structure.

Population genetics (16)

Allele frequency under haploid (genic) selection — `bio_haploid_selection`

The frequency of allele A after t generations of haploid selection (relative fitness 1 + s against 1).

$$p' = \frac{p(1+s)}{1+sp}$$

symbol		meaning	SI unit
`p0`	in	initial frequency of A	—
`s`	in	selection coefficient (fitness of A = 1 + s)	—
`t`	in	number of generations	—
`p_t`	out	frequency of A after t generations	—

Applies when: 0 < p0 < 1; s > -1; discrete generations.

```
$ computational-biology solve bio_haploid_selection p0=0.5 s=1 t=1
p_t = 0.6666666667
  by: Allele frequency under haploid (genic) selection  (iterate p' = p(1+s)/(1+sp) generation by generation)
  p0 = 0.5
  s = 1
  assumes: 0 < p0 < 1; s > -1; discrete generations
```

What the worked example means: The model calculates the frequency of allele A after a specified number of generations under haploid selection. Starting from an initial frequency of 0.5, with a selection coefficient of 1, the algorithm iterates the recurrence relation to determine the final state. The resulting frequency, 0.666667, represents the proportion of the population carrying allele A once evolutionary pressure has acted for 1 discrete time steps.

Decline of a recessive lethal allele — `bio_recessive_lethal`

The frequency of a recessive lethal allele after t generations of selection (aa never reproduce).

$$q' = \frac{q}{1+q}$$

symbol		meaning	SI unit
`q0`	in	initial frequency of the lethal allele	—
`t`	in	number of generations	—
`q_t`	out	frequency after t generations	—

Applies when: 0 < q0 <= 1; random mating, aa fitness 0.

```
$ computational-biology solve bio_recessive_lethal q0=0.5 t=1
q_t = 0.3333333333
  by: Decline of a recessive lethal allele  (iterate q' = q/(1+q))
  q0 = 0.5
  assumes: 0 < q0 <= 1; random mating, aa fitness 0
```

What the worked example means: This calculation determines how much a harmful recessive allele has declined in frequency over time. Starting from an initial frequency of 0.5, the model applies selection pressure where homozygous individuals do not reproduce, iterating the formula for 1 generations to yield a final frequency of 0.333333. This result illustrates that while selection removes these alleles efficiently when they are common, their decline slows as they become rare because most copies are hidden in healthy carriers.

Effective population size with unequal sex ratio — `bio_ne_sex_ratio`

The effective population size of a population with N_m breeding males and N_f breeding females.

$$N_e = \frac{4N_mN_f}{N_m + N_f}$$

symbol		meaning	SI unit
`N_m`	in	number of breeding males	—
`N_f`	in	number of breeding females	—
`Ne`	out	effective population size	—
`Ne_over_N`	out	Ne divided by the census N_m + N_f	—

Applies when: random mating within the breeding population; Wright-Fisher reproduction.

```
$ computational-biology solve bio_ne_sex_ratio N_m=10 N_f=90
Ne = 36
Ne_over_N = 0.36
  by: Effective population size with unequal sex ratio  (Wright's formula)
  assumes: random mating within the breeding population; Wright-Fisher reproduction
```

What the worked example means: This calculation determines the effective population size using Wright's formula for unequal sex ratios. It takes the number of breeding males, 10, and breeding females, 90, to compute the genetic contribution balance. The resulting effective size, 36, reflects how many individuals would be needed in an idealized population to experience the same rate of genetic drift. This value represents 0.36 of the actual census size, highlighting the impact of sex ratio imbalance on genetic diversity.

Effective size of a fluctuating population — `bio_ne_fluctuating_size`

The effective population size over generations of changing size (harmonic mean) and the fraction of heterozygosity retained over those generations.

$$\frac{1}{N_e} = \frac{1}{t}\sum_{i=1}^{t}\frac{1}{N_i}, \quad \frac{H_t}{H_0} = \prod_i\left(1 - \frac{1}{2N_i}\right)$$

symbol		meaning	SI unit
`sizes`	in	numbers of diploid individuals in successive generations	—
`Ne`	out	effective size (harmonic mean of the sizes)	—
`retained`	out	heterozygosity retained, product of (1 - 1/(2N_i))	—

Applies when: whole positive sizes; Wright-Fisher reproduction each generation.

```
$ computational-biology solve bio_ne_fluctuating_size "sizes=[1000, 10, 1000]"
Ne = 29.41176471
retained = 0.9490502375
  by: Effective size of a fluctuating population  (harmonic mean of the generation sizes; heterozygosity falls by 1 - 1/(2N_i) in
generation i)
  assumes: whole positive sizes; Wright-Fisher reproduction each generation
```

What the worked example means: This calculation determines the effective population size, which is the harmonic mean of the individual generation sizes provided in [1000, 10, 1000]. It also computes the fraction of initial genetic variation that remains after these generations, shown as 0.94905. The result highlights how brief reductions in population count disproportionately impact genetic diversity compared to periods of growth.

Equilibrium under heterozygote advantage — `bio_heterozygote_advantage`

The stable equilibrium frequency of allele A when the heterozygote is fittest (overdominance).

$$\hat{p} = \frac{w_{Aa} - w_{aa}}{2w_{Aa} - w_{AA} - w_{aa}} = \frac{t}{s + t}$$

symbol		meaning	SI unit
`w_AA`	in	relative fitness of AA	—
`w_Aa`	in	relative fitness of Aa	—
`w_aa`	in	relative fitness of aa	—
`p_eq`	out	equilibrium frequency of A	—

Applies when: w_Aa > w_AA and w_Aa > w_aa; fitnesses positive.

```
$ computational-biology solve bio_heterozygote_advantage w_AA=0.8 w_Aa=1 w_aa=0.6
p_eq = 0.6666666667
  by: Equilibrium under heterozygote advantage  (the interior fixed point of the selection recursion)
  w_AA = 0.8
  w_Aa = 1
  w_aa = 0.6
  assumes: w_Aa > w_AA and w_Aa > w_aa; fitnesses positive
```

What the worked example means: This calculation finds the stable genetic state where the allele frequency stops changing because the mixed genotype survives better than either pure type. Given the relative fitnesses of the two homozygotes and the heterozygote, the model computes that allele A will settle at a frequency of 0.666667. This result indicates how natural selection maintains diversity by favoring individuals carrying both versions of the gene over those with just one kind.

Expected coalescence times — `bio_coalescent_times`

The expected time to the most recent common ancestor and the expected total branch length of the genealogy of n gene copies from a diploid population of effective size Ne (in generations).

$$E[T_{MRCA}] = 4N_e\left(1 - \frac{1}{n}\right), \quad E[L] = 4N_e\sum_{i=1}^{n-1}\frac{1}{i}$$

symbol		meaning	SI unit
`n`	in	number of sampled gene copies	—
`Ne`	in	diploid effective population size	—
`t_mrca`	out	expected TMRCA in generations	—
`total_length`	out	expected total branch length in generations	—

Applies when: neutral Kingman coalescent; diploid: a pair coalesces at rate 1/(2Ne) per generation.

```
$ computational-biology solve bio_coalescent_times n=2 Ne=1000
t_mrca = 2000
total_length = 4000
  by: Expected coalescence times  (sum of exponential waiting times while k lineages remain, mean 4Ne/(k(k-1)))
  Ne = 1000
  assumes: neutral Kingman coalescent; diploid: a pair coalesces at rate 1/(2Ne) per generation
```

What the worked example means: This calculation determines the expected time to the most recent common ancestor and the total branch length for a sample of 2 gene copies drawn from a diploid population with an effective size of 1000. Under the neutral Kingman coalescent model, these values represent the average duration in generations until all sampled lineages merge into one ancestral lineage, and the sum of all branch lengths in that genealogy. The result shows that the expected time to the most recent common ancestor is 2000 generations, while the expected total branch length is 4000 generations, which are key metrics for estimating mutation accumulation and demographic history.

Fixation index FST from subpopulation allele frequencies — `bio_fst`

Mean within-subpopulation heterozygosity HS, total heterozygosity HT and FST = (HT - HS)/HT for a biallelic locus in equally weighted subpopulations.

$$F_{ST} = \frac{H_T - H_S}{H_T}$$

symbol		meaning	SI unit
`p`	in	frequency of one allele in each subpopulation	—
`HS`	out	mean expected heterozygosity within subpopulations	—
`HT`	out	expected heterozygosity of the pooled population	—
`FST`	out	fixation index (HT - HS)/HT	—

Applies when: equal subpopulation weights; biallelic locus; 0 < mean p < 1.

```
$ computational-biology solve bio_fst "p=[0.2, 0.8]"
HS = 0.32
HT = 0.5
FST = 0.36
  by: Fixation index FST from subpopulation allele frequencies  (HS = mean of 2p(1-p), HT = 2 pbar (1 - pbar), FST = (HT - HS)/HT)
  assumes: equal subpopulation weights; biallelic locus; 0 < mean p < 1
```

What the worked example means: This calculation determines the fixation index by comparing genetic diversity within groups to the overall population. It uses the mean expected heterozygosity within subpopulations, denoted as 0.32, and the expected heterozygosity of the pooled population, denoted as 0.5. The resulting fixation index, 0.36, represents the proportion of total genetic variation attributable to differences between subpopulations. A higher value indicates greater genetic differentiation among the groups.

Fixation probability in the Moran process — `bio_moran_fixation`

The probability that a mutant with relative fitness r, starting from i copies in a Moran population of size N, eventually takes over the population.

$$\rho_i = \frac{1 - r^{-i}}{1 - r^{-N}}$$

symbol		meaning	SI unit
`N`	in	population size	—
`i`	in	initial number of mutants	—
`r`	in	relative fitness of the mutant	—
`rho`	out	fixation probability	—

Applies when: 0 < i < N; r > 0.

```
$ computational-biology solve bio_moran_fixation N=10 i=1 r=1
rho = 0.1
  by: Fixation probability in the Moran process  (gambler's-ruin solution of the Moran birth-death chain (i/N when r = 1))
  r = 1
  assumes: 0 < i < N; r > 0
```

What the worked example means: This calculation determines the likelihood that a specific mutant variant will eventually replace all other variants in a fixed-size population. Using the gambler's ruin solution for the Moran birth-death chain, it computes the fixation probability based on the initial number of mutants, the total population size, and the mutant's relative fitness. The resulting value of 0.1 represents the chance of evolutionary takeover under these stochastic conditions.

Hardy-Weinberg chi-square test — `bio_hardy_weinberg_test`

Allele frequencies, Hardy-Weinberg expected genotype counts, the chi-square test of equilibrium (1 degree of freedom), observed and expected heterozygosity and the inbreeding coefficient F.

$$\chi^2 = \sum \frac{(O - E)^2}{E}, \quad E = (Np^2,\ 2Npq,\ Nq^2)$$

symbol		meaning	SI unit
`n_AA`	in	number of AA individuals	—
`n_Aa`	in	number of Aa individuals	—
`n_aa`	in	number of aa individuals	—
`p`	out	frequency of allele A	—
`q`	out	frequency of allele a	—
`exp_AA`	out	expected AA count (N p^2)	—
`exp_Aa`	out	expected Aa count (2 N p q)	—
`exp_aa`	out	expected aa count (N q^2)	—
`chi2`	out	chi-square statistic	—
`p_value`	out	p-value (chi-square, 1 degree of freedom)	—
`Ho`	out	observed heterozygosity	—
`He`	out	expected heterozygosity 2pq	—
`F`	out	inbreeding coefficient 1 - Ho/He	—

Applies when: both alleles present; biallelic locus; no continuity correction.

```
$ computational-biology solve bio_hardy_weinberg_test n_AA=298 n_Aa=489 n_aa=213
p = 0.5425
q = 0.4575
exp_AA = 294.30625
exp_Aa = 496.3875
exp_aa = 209.30625
chi2 = 0.2214895876
p_value = 0.6379072638
Ho = 0.489
He = 0.4963875
F = 0.01488252625
  by: Hardy-Weinberg chi-square test  (allele counting, expected counts N p^2, 2Npq, N q^2, chi-square with 3 - 1 - 1 = 1 degree
of freedom)
  assumes: both alleles present; biallelic locus; no continuity correction
```

What the worked example means: This calculation determines whether observed genotype counts differ significantly from those expected under Hardy-Weinberg equilibrium. Starting with input counts for homozygous dominant, heterozygous, and homozygous recessive individuals, the method computes allele frequencies p and q. These frequencies generate expected genotype counts based on the product of total population size and Hardy-Weinberg proportions. The chi-square statistic compares observed and expected values, resulting in a value of 0.22149. A corresponding p-value of 0.637907 assesses statistical significance. Additionally, the analysis reports observed heterozygosity at 0.489, expected heterozygosity at 0.496387, and an inbreeding coefficient F of 0.0148825, quantifying the deficit or excess of heterozygotes relative to random mating expectations.

Hardy-Weinberg exact test — `bio_hardy_weinberg_exact_test`

The exact (Wigginton) test of Hardy-Weinberg equilibrium for a biallelic locus: the two-sided p-value and the probability of the observed number of heterozygotes given the allele counts.

$$P(n_{AB} \mid n_A) = \frac{N!}{n_{AA}!\, n_{AB}!\, n_{BB}!} \, \frac{2^{n_{AB}}\, n_A!\, n_B!}{(2N)!}$$

symbol		meaning	SI unit
`n_AA`	in	number of AA individuals	—
`n_Aa`	in	number of Aa individuals	—
`n_aa`	in	number of aa individuals	—
`p_value`	out	exact two-sided p-value (sum of probabilities not above the observed one)	—
`p_observed`	out	probability of the observed heterozygote count	—

Applies when: both alleles present; N <= 1000.

```
$ computational-biology solve bio_hardy_weinberg_exact_test n_AA=2 n_Aa=0 n_aa=2
p_value = 0.08571428571
p_observed = 0.08571428571
  by: Hardy-Weinberg exact test  (exact conditional distribution of heterozygote counts given the allele counts (rational
arithmetic))
  assumes: both alleles present; N <= 1000
```

What the worked example means: This calculation determines the exact probability of observing the specific number of heterozygotes given the total allele counts in the sample. Using rational arithmetic to avoid approximation errors, it computes the likelihood of the observed genotype configuration compared to all possible configurations with the same allele frequencies. The resulting p-value of 0.0857143 quantifies how likely such a deviation from expected proportions would occur by chance if the population were truly in equilibrium. A low value suggests significant departure from random mating assumptions.

Linkage disequilibrium D, D' and r^2 — `bio_linkage_disequilibrium`

The linkage disequilibrium between two biallelic loci from haplotype counts: D, Lewontin's D' and r^2.

$$D = p_{AB} - p_A p_B, \quad r^2 = \frac{D^2}{p_A q_A p_B q_B}, \quad D' = \frac{D}{D_{\max}}$$

symbol		meaning	SI unit
`n_AB`	in	number of AB haplotypes	—
`n_Ab`	in	number of Ab haplotypes	—
`n_aB`	in	number of aB haplotypes	—
`n_ab`	in	number of ab haplotypes	—
`D`	out	disequilibrium D = pAB - pA pB	—
`D_prime`	out	Lewontin's D' = D / Dmax (signed)	—
`r2`	out	squared correlation r^2	—

Applies when: both loci polymorphic; phased haplotype counts.

```
$ computational-biology solve bio_linkage_disequilibrium n_AB=40 n_Ab=10 n_aB=10 n_ab=40
D = 0.15
D_prime = 0.6
r2 = 0.36
  by: Linkage disequilibrium D, D' and r^2  (haplotype and allele frequencies (exact fractions); Dmax from the allele frequencies)
  assumes: both loci polymorphic; phased haplotype counts
```

What the worked example means: This calculation determines the linkage disequilibrium between two genetic loci using phased haplotype counts. The raw disequilibrium coefficient D equals 0.15, representing the difference between observed and expected haplotype frequencies. To account for allele frequency constraints, this value is normalized to produce Lewontin's D prime at 0.6 and the squared correlation coefficient r squared at 0.36, which indicate the strength of non-random association.

Loss of heterozygosity by genetic drift — `bio_drift_heterozygosity`

The expected heterozygosity after t generations of genetic drift in a Wright-Fisher population of N diploids, starting from i copies of an allele.

$$H_t = H_0\left(1 - \frac{1}{2N}\right)^t$$

symbol		meaning	SI unit
`N`	in	number of diploid individuals	—
`i`	in	initial copies of allele A (of 2N)	—
`t`	in	number of generations	—
`H0`	out	initial heterozygosity 2pq	—
`H_t`	out	expected heterozygosity after t generations	—
`retained`	out	fraction of heterozygosity retained, (1 - 1/(2N))^t	—

Applies when: 0 < i < 2N; constant N, random mating, no mutation or selection.

```
$ computational-biology solve bio_drift_heterozygosity N=10 i=10 t=1
H0 = 0.5
H_t = 0.475
retained = 0.95
  by: Loss of heterozygosity by genetic drift  (expected heterozygosity falls by the factor 1 - 1/(2N) each generation)
  assumes: 0 < i < 2N; constant N, random mating, no mutation or selection
```

What the worked example means: This calculation determines the expected level of genetic variation remaining in a population after a specific period of genetic drift. Starting with an initial heterozygosity of 0.5, derived from 10 copies of an allele within a population of 10 diploid individuals, the model projects that variation declines over 1 generations. The final expected heterozygosity is 0.475, which corresponds to retaining 0.95 of the original genetic diversity, illustrating how finite population size erodes variability through random sampling alone.

Mutation-selection balance — `bio_mutation_selection_balance`

The equilibrium frequency of a deleterious allele maintained by recurrent mutation against selection (fitness 1, 1 - hs, 1 - s), from the exact recursion.

$$\hat{q} \approx \sqrt{\mu/s}\ (h = 0), \quad \hat{q} \approx \frac{\mu}{hs}\ (h > 0)$$

symbol		meaning	SI unit
`mu`	in	mutation rate A -> a per generation	—
`s`	in	selection coefficient against aa	—
`h`	in	dominance coefficient (0 recessive, 0.5 additive)	—
`q_eq`	out	equilibrium frequency of the deleterious allele	—

Applies when: 0 < mu <= 0.01; 0.01 <= s <= 1; 0 <= h <= 1; selection then mutation, no back mutation.

```
$ computational-biology solve bio_mutation_selection_balance mu=0.0001 s=1 h=0
q_eq = 0.01
  by: Mutation-selection balance  (the root of q' - q = 0 for selection followed by mutation, by bisection (approximately
sqrt(mu/s) for a recessive and mu/(hs) otherwise))
  mu = 0.0001
  s = 1
  h = 0
  assumes: 0 < mu <= 0.01; 0.01 <= s <= 1; 0 <= h <= 1; selection then mutation, no back mutation
```

What the worked example means: This calculation determines the stable frequency of a harmful gene variant in a population where new mutations constantly arise but natural selection removes them. Solving for the point where the change in allele frequency due to selection followed by mutation equals zero shows that the equilibrium frequency is approximately 0.01. This result indicates how often the deleterious trait persists despite being disadvantageous, offering insight into genetic load and disease prevalence under the given parameters of mutation rate, selection coefficient, and dominance.

Nucleotide diversity — `bio_nucleotide_diversity`

The average number of pairwise differences k and the nucleotide diversity pi (per site) of a set of aligned DNA sequences.

$$\pi = \frac{1}{L}\binom{n}{2}^{-1}\sum_{i<j} d_{ij}$$

symbol		meaning	SI unit
`seqs`	in	aligned DNA sequences separated by spaces	—
`k`	out	mean number of pairwise differences	—
`pi`	out	nucleotide diversity per site	—

Applies when: at least 2 aligned DNA sequences of equal length, no gaps.

```
$ computational-biology solve bio_nucleotide_diversity "seqs=AAAA AAAT AATT"
k = 1.333333333
pi = 0.3333333333
  by: Nucleotide diversity  (average the differences over all pairs of sequences; divide by the alignment length)
  seqs = AAAA AAAT AATT
  assumes: at least 2 aligned DNA sequences of equal length, no gaps
```

What the worked example means: This calculation determines the average genetic variation within a set of aligned DNA sequences. It computes the mean number of differences between all possible pairs of sequences, resulting in a value of 1.33333, and then normalizes this by the sequence length to provide the nucleotide diversity per site, which is 0.333333. These metrics indicate how genetically distinct the sampled individuals are from one another.

Tajima's D — `bio_tajima_d`

Tajima's D neutrality statistic of aligned DNA sequences (the scaled difference between pi and Watterson's theta).

$$D = \frac{\hat{\theta}_\pi - \hat{\theta}_W}{\sqrt{e_1 S + e_2 S(S-1)}}$$

symbol		meaning	SI unit
`seqs`	in	aligned DNA sequences separated by spaces (at least 4)	—
`D`	out	Tajima's D	—
`k`	out	mean number of pairwise differences	—

Applies when: at least 4 aligned DNA sequences; at least one segregating site.

```
$ computational-biology solve bio_tajima_d "seqs=AAAAAA AAAAAT AAAATT AAATTT"
D = 0.167655795
k = 1.666666667
  by: Tajima's D  (D = (k - S/a1) / sqrt(e1 S + e2 S (S-1)) with Tajima's constants)
  seqs = AAAAAA AAAAAT AAAATT AAATTT
  assumes: at least 4 aligned DNA sequences; at least one segregating site
```

What the worked example means: This calculation determines Tajima's D, a statistic that compares two estimates of genetic diversity. It uses the average number of pairwise nucleotide differences, represented here as 1.66667, and contrasts it with Watterson's estimator based on segregating sites. The resulting value of 0.167656 indicates whether the population deviates from neutral evolution expectations, where positive values suggest balancing selection or population contraction, and negative values imply recent expansion or purifying selection.

Watterson's estimator of theta — `bio_watterson_theta`

The number of segregating sites S and Watterson's estimator of the population mutation rate theta, per sequence and per site.

$$\hat{\theta}_W = \frac{S}{a_n}, \quad a_n = \sum_{i=1}^{n-1} \frac{1}{i}$$

symbol		meaning	SI unit
`seqs`	in	aligned DNA sequences separated by spaces	—
`S`	out	number of segregating sites	—
`theta`	out	Watterson's theta per sequence (S / a_n)	—
`theta_site`	out	Watterson's theta per site	—

Applies when: at least 2 aligned DNA sequences of equal length, no gaps.

```
$ computational-biology solve bio_watterson_theta "seqs=AAAA AAAT AATT"
S = 2
theta = 1.333333333
theta_site = 0.3333333333
  by: Watterson's estimator of theta  (count segregating sites and divide by the harmonic number a_n = sum_{i=1}^{n-1} 1/i)
  seqs = AAAA AAAT AATT
  assumes: at least 2 aligned DNA sequences of equal length, no gaps
```

What the worked example means: This calculation determines the population mutation rate using Watterson's estimator. It counts the number of variable positions in the aligned sequences, denoted as S, which equals 2. This count is divided by the harmonic number derived from the sample size to yield theta per sequence, calculated as 1.33333. To normalize for sequence length, this value is further divided by the total number of sites to produce theta per site, shown as 0.333333. These metrics provide a baseline expectation for genetic diversity under neutral evolution.

Sequences (21)

Amino-acid composition of a protein — `bio_amino_acid_composition`

The percentage of each of the 20 standard amino acids in a protein, its most common residue (alphabetically first among ties) and its length.

$$\%a = 100\,\frac{n_a}{n}$$

symbol		meaning	SI unit
`seq`	in	protein sequence	—
`percent`	out	percentage of each amino acid (one-letter code -> %)	—
`most_common`	out	most common amino acid (alphabetically first among ties)	—
`length`	out	number of residues	—

Applies when: standard amino acids only.

```
$ computational-biology solve bio_amino_acid_composition seq=MAAAK
percent =
{"A":60.0,"C":0.0,"D":0.0,"E":0.0,"F":0.0,"G":0.0,"H":0.0,"I":0.0,"K":20.0,"L":0.0,"M":20.0,"N":0.0,"P":0.0,"Q":0.0,"R":0.0,"S":0.0,"T":0.0,"V"
most_common = A
length = 5
  by: Amino-acid composition of a protein  (count each residue and divide by the length)
  seq = MAAAK
  assumes: standard amino acids only
```

What the worked example means: This calculation determines the percentage composition of each standard amino acid within the provided protein sequence. By counting every residue and dividing by the total number of residues, it reveals that the sequence consists of MAAAK characters. The analysis identifies A as the most frequently occurring residue, while the total length is recorded as 5. This breakdown helps characterize the chemical nature of the protein based on its primary structure.

Aromaticity of a protein — `bio_aromaticity`

The aromaticity (relative frequency of Phe + Trp + Tyr, Lobry 1994) of a protein and its number of aromatic residues.

$$\mathrm{Ar} = \frac{n_F + n_W + n_Y}{n}$$

symbol		meaning	SI unit
`seq`	in	protein sequence	—
`aromaticity`	out	fraction of F, W and Y residues	—
`aromatic_count`	out	number of F, W and Y residues	—

Applies when: standard amino acids only.

```
$ computational-biology solve bio_aromaticity seq=MFWYAK
aromaticity = 0.5
aromatic_count = 3
  by: Aromaticity of a protein  (count F, W and Y and divide by the length)
  seq = MFWYAK
  assumes: standard amino acids only
```

What the worked example means: This calculation determines the aromaticity of a protein sequence by identifying phenylalanine, tryptophan, and tyrosine residues. The result, expressed as 0.5, represents the fraction of these specific amino acids relative to the total length of the input sequence MFWYAK. This metric provides insight into the hydrophobic character and potential stability contributions of the protein structure.

Edit (Levenshtein) distance between sequences — `bio_edit_distance`

The minimum number of substitutions, insertions and deletions turning one sequence into the other (unit costs; an unaligned comparison, unlike the p-distance).

$$D_{i,j} = \min(D_{i-1,j} + 1,\ D_{i,j-1} + 1,\ D_{i-1,j-1} + [x_i \neq y_j])$$

symbol		meaning	SI unit
`seq1`	in	first sequence	—
`seq2`	in	second sequence	—
`distance`	out	edit distance (unit costs)	—

Applies when: letters only.

```
$ computational-biology solve bio_edit_distance seq1=KITTEN seq2=SITTING
distance = 3
  by: Edit (Levenshtein) distance between sequences  (dynamic programming over prefixes (Wagner-Fischer))
  seq1 = KITTEN
  seq2 = SITTING
  assumes: letters only
```

What the worked example means: This calculation determines the minimum number of single-character edits, including insertions, deletions, or substitutions, required to transform the first sequence into the second. The result, represented as 3, quantifies the dissimilarity between KITTEN and SITTING under the assumption that all these operations carry a uniform cost of one unit.

GC content of a nucleic acid — `bio_gc_content`

The fraction and number of G and C bases in a DNA or RNA sequence.

$$\mathrm{GC} = \frac{n_G + n_C}{n}$$

symbol		meaning	SI unit
`seq`	in	DNA or RNA sequence	—
`gc_fraction`	out	fraction of G + C	—
`gc_count`	out	number of G and C bases	—

Applies when: unambiguous DNA or RNA letters.

```
$ computational-biology solve bio_gc_content seq=ATGCGCGA
gc_fraction = 0.625
gc_count = 5
  by: GC content of a nucleic acid  (count G and C and divide by the length)
  seq = ATGCGCGA
  assumes: unambiguous DNA or RNA letters
```

What the worked example means: This calculation determines the proportion of guanine and cytosine bases within the provided nucleic acid sequence. Counting the occurrences of these specific bases, which totals 5, and dividing by the total length of the sequence gives a GC fraction of 0.625. This metric serves as an indicator of the thermal stability and structural properties of the DNA or RNA molecule.

Grand average of hydropathy (GRAVY) — `bio_gravy`

The Kyte-Doolittle grand average of hydropathy of a protein (positive = hydrophobic).

$$\mathrm{GRAVY} = \frac{1}{n}\sum_{i} h(a_i)$$

symbol		meaning	SI unit
`seq`	in	protein sequence	—
`gravy`	out	GRAVY score	—

Applies when: standard amino acids only.

```
$ computational-biology solve bio_gravy seq=MKTLLIL
gravy = 1.885714286
  by: Grand average of hydropathy (GRAVY)  (mean Kyte-Doolittle hydropathy over the residues)
  seq = MKTLLIL
  assumes: standard amino acids only
```

What the worked example means: This calculation determines the Grand Average of Hydropathy by taking the mean of Kyte-Doolittle hydropathy indices for all residues in the protein sequence MKTLLIL. The resulting value, 1.88571, represents the average hydrophobicity of the entire chain. A positive value suggests a tendency toward hydrophobic interactions, often associated with membrane-spanning regions or buried cores, while a negative value indicates hydrophilic character typical of solvent-exposed surfaces. This metric provides a global summary of the sequence's chemical nature rather than identifying specific local motifs.

Isoelectric point of a protein — `bio_isoelectric_point`

The isoelectric point (pH of zero net charge) of a protein, with its net charge at pH 7 (Bjellqvist pK set).

$$Z(\mathrm{pI}) = 0$$

symbol		meaning	SI unit
`seq`	in	protein sequence	—
`pI`	out	isoelectric point	—
`charge_at_7`	out	net charge at pH 7	—

Applies when: standard amino acids only.

```
$ computational-biology solve bio_isoelectric_point seq=INGAR
pI = 9.750024638
charge_at_7 = 0.7600916143
  by: Isoelectric point of a protein  (the net charge decreases with pH: bisection for the pH where it is zero, to full precision)
  seq = INGAR
  assumes: standard amino acids only
```

What the worked example means: This calculation determines the specific pH value where the protein carries no overall electrical charge, known as the isoelectric point. The result is derived from the amino acid sequence by summing the charges of all ionizable groups using standard pK values. At the computed isoelectric point of 9.75002, the positive and negative charges balance perfectly. Additionally, the net charge at a physiological pH of seven is reported as 0.760092, indicating whether the molecule tends to attract or repel other charged species in typical biological environments.

k-mer counts of a sequence — `bio_kmer_counts`

Overlapping k-mer counts: the number of k-mers, of distinct k-mers, the highest count and the most frequent k-mer (alphabetically first among ties).

$$N_k = n - k + 1$$

symbol		meaning	SI unit
`seq`	in	DNA or RNA sequence	—
`k`	in	k-mer length	—
`total`	out	number of overlapping k-mers (n - k + 1)	—
`distinct`	out	number of distinct k-mers	—
`max_count`	out	count of the most frequent k-mer	—
`most_frequent`	out	most frequent k-mer (alphabetically first among ties)	—

Applies when: 1 <= k <= sequence length.

```
$ computational-biology solve bio_kmer_counts seq=ATATAT k=2
total = 5
distinct = 2
max_count = 3
most_frequent = AT
  by: k-mer counts of a sequence  (slide a window of length k one base at a time and count each word)
  seq = ATATAT
  assumes: 1 <= k <= sequence length
```

What the worked example means: This calculation determines the frequency of every substring of length k within the provided nucleotide sequence. Sliding a window of size 2 across the sequence ATATAT shows that there are 5 overlapping k-mers in total. Among these, 2 unique patterns appear. The most common pattern observed is AT, which occurs 3 times.

Longest open reading frame — `bio_longest_orf`

The longest open reading frame (ATG to the first in-frame stop, stop included) over all six frames of a DNA sequence, its protein length and the number of ORFs.

$$\ell = 3(j - i + 1) \text{ for ATG at codon } i, \text{ first stop at codon } j$$

symbol		meaning	SI unit
`seq`	in	DNA sequence 5'->3'	—
`length_nt`	out	length of the longest ORF in nucleotides, stop codon included	—
`protein_length`	out	amino acids encoded by the longest ORF (stop excluded)	—
`orf_count`	out	number of ORFs (one per stop codon reached from an upstream in-frame ATG)	—

Applies when: DNA letters only; standard genetic code, start ATG, stops TAA TAG TGA.

```
$ computational-biology solve bio_longest_orf seq=CCATGAAATTTGGGTAACC
length_nt = 15
protein_length = 4
orf_count = 1
  by: Longest open reading frame  (scan the three frames of both strands codon by codon: open at the first ATG, close at the next
stop)
  seq = CCATGAAATTTGGGTAACC
  assumes: DNA letters only; standard genetic code, start ATG, stops TAA TAG TGA
```

What the worked example means: This calculation identifies the longest continuous coding sequence within the provided DNA strand by scanning all six possible reading frames. It begins at the first start codon and ends at the first in-frame stop codon. The resulting longest open reading frame measures 15 nucleotides, which corresponds to a protein length of 4 amino acids when excluding the stop signal. Additionally, the total number of valid open reading frames detected across the sequence is 1.

Melting temperature by nearest-neighbour thermodynamics — `bio_tm_nearest_neighbor`

The melting temperature of a DNA duplex (perfect complement) from nearest-neighbour thermodynamics (Allawi-SantaLucia 1997) with the SantaLucia 1998 sodium correction.

$$T_m = \frac{1000\,\Delta H}{\Delta S + 0.368(N-1)\ln[\mathrm{Na^+}] + R\ln k} - 273.15$$

symbol		meaning	SI unit
`seq`	in	DNA sequence 5'->3'	—
`Na`	in	sodium concentration in mM	—
`dnac1`	in	concentration of the more concentrated strand in nM	—
`dnac2`	in	concentration of the less concentrated strand in nM	—
`tm`	out	melting temperature in degC	—

Applies when: unambiguous DNA, length >= 2; Na > 0; dnac1 > dnac2/2.

```
$ computational-biology solve bio_tm_nearest_neighbor seq=CGTTCCAAAGATGTGGGCATGAGCTTAC Na=50 dnac1=25 dnac2=25
tm = 60.3209195
  by: Melting temperature by nearest-neighbour thermodynamics  (sum nearest-neighbour enthalpies and entropies with initiation
terms; Tm = 1000 dH / (dS + R ln k) - 273.15 with dS corrected by 0.368 (N-1) ln[Na+])
  Na = 50
  dnac1 = 25
  dnac2 = 25
  seq = CGTTCCAAAGATGTGGGCATGAGCTTAC
  assumes: unambiguous DNA, length >= 2; Na > 0; dnac1 > dnac2/2
```

What the worked example means: This calculation determines the melting temperature of a DNA duplex using nearest-neighbor thermodynamics. It sums the enthalpy and entropy contributions from adjacent base pairs, including initiation terms, and applies corrections for sodium ion concentration and strand stoichiometry. The resulting value, 60.3209, represents the temperature at which half of the DNA strands are dissociated under these specific conditions.

Melting temperature by the Wallace rule — `bio_tm_wallace`

The approximate melting temperature of a short DNA primer (14-20 nt) by the Wallace rule.

$$T_m = 2(n_A + n_T) + 4(n_G + n_C)\ ^\circ\mathrm{C}$$

symbol		meaning	SI unit
`seq`	in	DNA primer sequence	—
`tm`	out	melting temperature in degC	—

Applies when: unambiguous DNA letters.

```
$ computational-biology solve bio_tm_wallace seq=ACGTTGCAATGCCGTA
tm = 48
  by: Melting temperature by the Wallace rule  (2 degC per A or T plus 4 degC per G or C)
  seq = ACGTTGCAATGCCGTA
  assumes: unambiguous DNA letters
```

What the worked example means: This calculation determines the approximate melting temperature of a short DNA primer using the Wallace rule. By counting the number of adenine and thymine bases, which contribute two degrees each, and guanine and cytosine bases, which contribute four degrees each, the tool derives the stability threshold. For the input sequence ACGTTGCAATGCCGTA, the resulting melting temperature is 48, indicating the point at which half of the primer strands are dissociated from their targets under standard conditions.

Molecular mass of a protein — `bio_protein_mass`

The average molecular mass of a protein from its amino-acid sequence.

$$M = \sum_i m(a_i) - (n-1)\, m_{\mathrm{H_2O}}$$

symbol		meaning	SI unit
`seq`	in	protein sequence (20 standard one-letter codes)	—
`mass`	out	molecular mass in g/mol (Da)	—

Applies when: standard amino acids only.

```
$ computational-biology solve bio_protein_mass
seq=MAEGEITTFTALTEKFNLPPGNYKKPKLLYCSNGGHFLRILPDGTVDGTRDRSDQHIQLQLSAESVGEVYIKSTETGQYLAMDTSGLLYGSQTPSEECLFLERLEENHYNTYTSKKHAEKNWFVGLKKNGSCKRGPRTH
mass = 17103.1617
  by: Molecular mass of a protein  (sum of amino-acid masses minus one water per peptide bond)
  seq =
MAEGEITTFTALTEKFNLPPGNYKKPKLLYCSNGGHFLRILPDGTVDGTRDRSDQHIQLQLSAESVGEVYIKSTETGQYLAMDTSGLLYGSQTPSEECLFLERLEENHYNTYTSKKHAEKNWFVGLKKNGSCKRGPRTHYGQK
  assumes: standard amino acids only
```

What the worked example means: This calculation determines the average molecular mass of a protein based on its amino acid sequence. It sums the masses of individual amino acids and subtracts the mass of water molecules lost during peptide bond formation. The result, expressed as 17103.2, represents the total mass per mole of the protein in grams per mole or Daltons.

Molecular mass of DNA or RNA — `bio_nucleic_acid_mass`

The average molecular mass of a single- or double-stranded linear DNA or RNA with a 5' phosphate.

$$M = \sum_i m(x_i) - (n-1)\, m_{\mathrm{H_2O}}$$

symbol		meaning	SI unit
`seq`	in	DNA or RNA sequence	—
`molecule`	in	molecule type	—
`strands`	in	single- or double-stranded	—
`mass`	out	molecular mass in g/mol (Da)	—

Applies when: the sequence letters match the stated molecule.

```
$ computational-biology solve bio_nucleic_acid_mass seq=AGC molecule=DNA strands=single
mass = 949.6095
  by: Molecular mass of DNA or RNA  (sum of nucleotide-monophosphate masses minus one water per phosphodiester bond (each strand))
  strands = single
  seq = AGC
  molecule = DNA
  assumes: the sequence letters match the stated molecule
```

What the worked example means: This calculation determines the average molecular mass of a DNA or RNA molecule based on its nucleotide composition. It sums the individual masses of each nucleotide monophosphate and subtracts the mass of water molecules lost during the formation of phosphodiester bonds between adjacent nucleotides. The final result, 949.61, represents the total mass per mole of the specified strand configuration, accounting for the presence of a five-prime phosphate group.

Net charge of a protein at a pH — `bio_protein_net_charge`

The net charge of a protein at a stated pH from its ionisable groups (Bjellqvist pK set).

$$Z = \sum_{\text{basic}} \frac{n_i}{1 + 10^{\,pH - pK_i}} - \sum_{\text{acidic}} \frac{n_j}{1 + 10^{\,pK_j - pH}}$$

symbol		meaning	SI unit
`seq`	in	protein sequence	—
`pH`	in	pH of the solution (0 to 14)	—
`charge`	out	net charge (elementary charges)	—

Applies when: standard amino acids only; 0 <= pH <= 14.

```
$ computational-biology solve bio_protein_net_charge seq=INGAR pH=7
charge = 0.7600916143
  by: Net charge of a protein at a pH  (Henderson-Hasselbalch partial charges of the termini and the K, R, H, D, E, C, Y side
chains)
  pH = 7
  seq = INGAR
  assumes: standard amino acids only; 0 <= pH <= 14
```

What the worked example means: This calculation determines the total electrical charge of a protein by summing the partial charges of its ionizable groups, including the termini and specific side chains. Using the provided sequence and a solution pH of 7, the model applies the Henderson-Hasselbalch equation to estimate how many groups are protonated or deprotonated. The resulting net charge is 0.760092, which indicates whether the protein carries an overall positive, negative, or neutral electrical state under these conditions.

p-distance (Hamming) between aligned sequences — `bio_hamming_distance`

Mismatches, p-distance and identity of two aligned sequences of equal length (no indels): the proportion of sites that differ.

$$p = \frac{1}{n} \sum_i [x_i \neq y_i]$$

symbol		meaning	SI unit
`seq1`	in	first aligned sequence	—
`seq2`	in	second aligned sequence (same length)	—
`mismatches`	out	number of differing sites	—
`p_distance`	out	proportion of differing sites	—
`identity`	out	proportion of identical sites	—

Applies when: equal lengths (aligned, no gaps).

```
$ computational-biology solve bio_hamming_distance seq1=GAGCCTACTAACGGGAT seq2=CATCGTAATGACGGCCT
mismatches = 7
p_distance = 0.4117647059
identity = 0.5882352941
  by: p-distance (Hamming) between aligned sequences  (compare the sequences site by site)
  seq1 = GAGCCTACTAACGGGAT
  seq2 = CATCGTAATGACGGCCT
  assumes: equal lengths (aligned, no gaps)
```

What the worked example means: This calculation compares the two provided aligned sequences site by site to determine their genetic difference. Counting the number of positions where the characters do not match gives 7. This count is then divided by the total number of sites in the sequences to yield the p-distance of 0.411765. Conversely, the identity, or proportion of matching sites, is 0.588235. This metric provides a simple measure of divergence assuming no insertions or deletions are present.

Protein extinction coefficient at 280 nm — `bio_protein_extinction`

The molar extinction coefficient at 280 nm (cysteines reduced, and all cysteines as cystines) and the absorbance of a 1 g/L solution.

$$\varepsilon_{280} = 5500\, n_W + 1490\, n_Y + 125\, n_{\text{cystine}}$$

symbol		meaning	SI unit
`seq`	in	protein sequence	—
`eps_reduced`	out	extinction coefficient, cysteines reduced (M^-1 cm^-1)	—
`eps_cystines`	out	extinction coefficient, cysteines paired as cystines (M^-1 cm^-1)	—
`abs_reduced`	out	absorbance of 1 g/L, 1 cm, cysteines reduced	—
`abs_cystines`	out	absorbance of 1 g/L, 1 cm, cystines	—

Applies when: standard amino acids only.

```
$ computational-biology solve bio_protein_extinction
seq=MAEGEITTFTALTEKFNLPPGNYKKPKLLYCSNGGHFLRILPDGTVDGTRDRSDQHIQLQLSAESVGEVYIKSTETGQYLAMDTSGLLYGSQTPSEECLFLERLEENHYNTYTSKKHAEKNWFVGLKKNGSCKRGPRTH
eps_reduced = 17420
eps_cystines = 17545
abs_reduced = 1.01852513
abs_cystines = 1.025833721
  by: Protein extinction coefficient at 280 nm  (5500 per Trp + 1490 per Tyr (+ 125 per cystine); divide by the molecular mass for
1 g/L)
  seq =
MAEGEITTFTALTEKFNLPPGNYKKPKLLYCSNGGHFLRILPDGTVDGTRDRSDQHIQLQLSAESVGEVYIKSTETGQYLAMDTSGLLYGSQTPSEECLFLERLEENHYNTYTSKKHAEKNWFVGLKKNGSCKRGPRTHYGQK
  assumes: standard amino acids only
```

In the worked example, seq = MAEGEITTFTALTEKFNLPPGNYKKPKLLYCSNGGHFLRILPDGTVDGTRDRSDQHIQLQLSAESVGEVYIKSTETGQYLAMDTSGLLYGSQTPSEECLFLERLEENHYNTYTSKKHAI go through the equation above and give eps_reduced = 17420, eps_cystines = 17545, abs_reduced = 1.01852513, abs_cystines = 1.025833721. It applies only when standard amino acids only: outside that it says what is wrong instead of returning a number.

Protein instability index — `bio_instability_index`

The Guruprasad instability index of a protein from its dipeptides (above 40 suggests an unstable protein in vivo).

$$II = \frac{10}{L} \sum_{i=1}^{L-1} \mathrm{DIWV}(a_i, a_{i+1})$$

symbol		meaning	SI unit
`seq`	in	protein sequence (at least 2 residues)	—
`index`	out	instability index	—

Applies when: standard amino acids, length >= 2.

```
$ computational-biology solve bio_instability_index
seq=MAEGEITTFTALTEKFNLPPGNYKKPKLLYCSNGGHFLRILPDGTVDGTRDRSDQHIQLQLSAESVGEVYIKSTETGQYLAMDTSGLLYGSQTPSEECLFLERLEENHYNTYTSKKHAEKNWFVGLKKNGSCKRGPRTH
index = 41.98026316
  by: Protein instability index  (sum of the dipeptide instability weights (DIWV) times 10/L)
  seq =
MAEGEITTFTALTEKFNLPPGNYKKPKLLYCSNGGHFLRILPDGTVDGTRDRSDQHIQLQLSAESVGEVYIKSTETGQYLAMDTSGLLYGSQTPSEECLFLERLEENHYNTYTSKKHAEKNWFVGLKKNGSCKRGPRTHYGQK
  assumes: standard amino acids, length >= 2
```

What the worked example means: This calculation determines the Guruprasad instability index by summing specific weight values assigned to each consecutive pair of amino acids in the provided protein sequence and normalizing this total by the sequence length. The resulting value, 41.9803, serves as a quantitative estimate of the protein's likely half-life within a cellular environment. An index exceeding forty generally suggests that the protein is unstable and prone to rapid degradation, whereas lower values imply greater stability. This metric relies solely on primary structure information to predict turnover rates without considering three-dimensional folding or external regulatory factors.

Restriction sites and fragments — `bio_restriction_sites`

Where a restriction enzyme cuts a linear DNA sequence: the number of sites, the cut positions and the fragment lengths.

$$\text{cut at } i + c \text{ for each site start } i$$

symbol		meaning	SI unit
`seq`	in	linear DNA sequence 5'->3'	—
`enzyme`	in	restriction enzyme (name)	—
`n_sites`	out	number of cuts	—
`positions`	out	cut positions (1-based first base after each cut, top strand)	—
`fragments`	out	fragment lengths in bp, 5'->3' order	—

Applies when: linear DNA; a listed enzyme.

```
$ computational-biology solve bio_restriction_sites seq=AAGAATTCTTGAATTCGGATCC enzyme=EcoRI
n_sites = 2
positions = [4,12]
fragments = [3,8,11]
  by: Restriction sites and fragments  (find every occurrence of the recognition site (overlaps included) and cut at the enzyme's
offset)
  enzyme = EcoRI
  seq = AAGAATTCTTGAATTCGGATCC
  assumes: linear DNA; a listed enzyme
```

In the worked example, seq = AAGAATTCTTGAATTCGGATCC, enzyme = EcoRI go through the equation above and give n_sites = 2, positions = [4, 12], fragments = [3, 8, 11]. It applies only when linear DNA; a listed enzyme: outside that it says what is wrong instead of returning a number.

Reverse complement of a DNA sequence — `bio_reverse_complement`

The reverse complement of a DNA strand (the partner strand read 5'->3').

$$5'\text{-}x_1 \dots x_n\text{-}3' \mapsto 5'\text{-}\bar{x}_n \dots \bar{x}_1\text{-}3'$$

symbol		meaning	SI unit
`seq`	in	DNA sequence 5'->3' (A, C, G, T)	—
`revcomp`	out	reverse complement 5'->3'	—

Applies when: DNA letters A, C, G, T only.

```
$ computational-biology solve bio_reverse_complement seq=ATGCCGTA
revcomp = TACGGCAT
  by: Reverse complement of a DNA sequence  (complement each base (A-T, C-G) and reverse the order)
  seq = ATGCCGTA
  assumes: DNA letters A, C, G, T only
```

What the worked example means: This calculation determines the sequence of the complementary DNA strand, oriented in the five-prime to three-prime direction. By taking the input sequence ATGCCGTA, each base is replaced by its pair (A with T, C with G) and the entire order is reversed. The resulting string TACGGCAT represents how the opposite strand would be read from its own five-prime end, which is essential for understanding antiparallel DNA structure.

Secondary-structure propensity fractions — `bio_secondary_structure_fraction`

The fractions of residues that tend to be in helices (E M A L K), turns (N P G S D) and sheets (V I Y F W L T), as ProtParam defines them (L counts for helix and sheet).

$$f_{\text{helix}} = \frac{n_E + n_M + n_A + n_L + n_K}{n}$$

symbol		meaning	SI unit
`seq`	in	protein sequence	—
`helix`	out	fraction of helix-favouring residues	—
`turn`	out	fraction of turn-favouring residues	—
`sheet`	out	fraction of sheet-favouring residues	—

Applies when: standard amino acids only.

```
$ computational-biology solve bio_secondary_structure_fraction seq=EMALKNPGSD
helix = 0.5
turn = 0.5
sheet = 0.1
  by: Secondary-structure propensity fractions  (count the residues of each propensity group)
  seq = EMALKNPGSD
  assumes: standard amino acids only
```

What the worked example means: This calculation determines the proportion of residues in the protein sequence that statistically favor forming specific secondary structures. By counting residues associated with helices, turns, and sheets according to ProtParam definitions, it yields the helix fraction as 0.5, the turn fraction as 0.5, and the sheet fraction as 0.1. These values provide a quick estimate of structural tendencies based solely on amino acid composition.

Transcription of DNA into mRNA — `bio_transcription`

The mRNA transcribed from a DNA strand given 5'->3', stated as the coding (sense) or template (antisense) strand.

$$\mathrm{mRNA} = \text{coding strand with } T \to U = \overline{\text{template}}^{\,R}$$

symbol		meaning	SI unit
`seq`	in	DNA strand 5'->3' (A, C, G, T)	—
`strand`	in	which strand is given: coding (sense) or template (antisense)	—
`mrna`	out	mRNA 5'->3'	—

Applies when: DNA letters only; the strand is stated.

```
$ computational-biology solve bio_transcription seq=ATGGCTTAA strand=coding
mrna = AUGGCUUAA
  by: Transcription of DNA into mRNA  (coding strand: T -> U; template strand: reverse complement, then T -> U)
  strand = coding
  seq = ATGGCTTAA
  assumes: DNA letters only; the strand is stated
```

In the worked example, seq = ATGGCTTAA, strand = coding go through the equation above and give mrna = AUGGCUUAA. It applies only when DNA letters only; the strand is stated: outside that it says what is wrong instead of returning a number.

Translation with the standard genetic code — `bio_translation`

The protein encoded by a DNA or mRNA sequence read in frame 1 with the standard genetic code; stop codons shown as '*' or the protein truncated at the first stop.

$$\text{codon } x_{3i+1}x_{3i+2}x_{3i+3} \mapsto \text{amino acid}_i$$

symbol		meaning	SI unit
`seq`	in	DNA or mRNA sequence 5'->3', length a multiple of 3	—
`stops`	in	show stop codons as '*' or truncate at the first stop	—
`protein`	out	amino-acid sequence (one-letter codes)	—
`length`	out	number of residues shown	—

Applies when: length a multiple of 3; DNA or RNA letters, not a mixture.

```
$ computational-biology solve bio_translation seq=ATGGCCATTGTAATGGGCCGCTGA stops=show
protein = MAIVMGR*
length = 8
  by: Translation with the standard genetic code  (read codons in frame 1 and look each up in the standard genetic code (NCBI
table 1))
  stops = show
  seq = ATGGCCATTGTAATGGGCCGCTGA
  assumes: length a multiple of 3; DNA or RNA letters, not a mixture
```

What the worked example means: This calculation translates the nucleic acid sequence ATGGCCATTGTAATGGGCCGCTGA into an amino acid sequence by reading triplet codons in frame one using the standard genetic code. The resulting protein is represented by one-letter codes and has a total length of 8 residues. If stop codons are present, they are marked as asterisks or the sequence is truncated depending on the setting for show, which determines how termination signals are displayed in the output.

Systems biology (1)

Flux balance analysis (optimal objective) — `bio_flux_balance_analysis`

The maximum of a linear objective (e.g. a biomass or product flux) over all steady-state flux distributions of a metabolic network within flux bounds: flux balance analysis.

$$\max_{v} \; c^{\mathsf{T}} v \;\text{ s.t. } Sv = 0,\;\; l \le v \le u$$

symbol		meaning	SI unit
`S`	in	stoichiometric matrix (metabolites x reactions)	—
`lb`	in	lower bound of each reaction flux	—
`ub`	in	upper bound of each reaction flux	—
`c`	in	objective coefficient of each reaction	—
`objective`	out	optimal objective value	—

Applies when: steady state S v = 0; finite bounds lb <= ub; a feasible flux distribution exists.

```
$ computational-biology solve bio_flux_balance_analysis "S=[[1, -1, 0, 0], [0, 1, -1, -1]]" "lb=[0, 0, 0, 0]" "ub=[10, 8, 100, 3]"
"c=[0, 0, 1, 0]"
objective = 8
  by: Flux balance analysis (optimal objective)  (linear programming solved exactly in rational arithmetic (two-phase simplex,
Bland's rule); the optimal value is unique even when the optimal fluxes are not)
  assumes: steady state S v = 0; finite bounds lb <= ub; a feasible flux distribution exists
```

What the worked example means: This calculation determines the maximum possible value for the specified biological objective, such as biomass production or product synthesis. It solves a linear programming problem where the objective coefficients are given by [0, 0, 1, 0] and the network structure is defined by the stoichiometric matrix [1 -1 0 0; 0 1 -1 -1]. The solution must satisfy mass balance constraints and respect the reaction limits set by lower bounds [0, 0, 0, 0] and upper bounds [10, 8, 100, 3]. The resulting optimal objective value is 8, which indicates the theoretical peak performance of the metabolic network under these conditions.

Questions and reports: support@ai999x.com. AI999X — The Future Is Self-Improving.